

The C Programming Language

From Syntax to Pointers

MS Roslyn

2026 年 10 月 9 日

摘要

本教程系统介绍 C 语言的语法与程序设计实践，涵盖程序结构与词法基础、数据类型、指针与地址、运算符与位运算、存储类别与内存模型、输入输出与文件操作，以及预处理与工程实践，并在附录中提供 C 标准库常用函数的速查手册。教程以简洁直观的方式组织知识，适合希望扎实掌握 C 语言原理与底层机制的读者。

目录

第一章 C 语言简介	7
1.1 诞生背景与历史地位	8
1.2 C 语言的特点	8
1.3 为什么学习 C 语言	9
1.4 从源代码到可执行文件	9
1.4.1 四个阶段	9
1.4.2 各阶段对照表	10
1.4.3 gcc 命令行示例	10
1.5 一个完整的 Hello World	10
1.5.1 逐行剖析	11
1.6 程序的基本结构	11
1.6.1 标识符与关键字	12
1.7 开发流程与常用工具	12
1.7.1 最小的编译运行示例	12
1.8 C 程序的内存模型概览	13
1.9 本章小结与常见误区	14
1.9.1 小结	14
1.9.2 常见误区	14
第二章 程序结构与词法基础	15
2.1 词法单元 (Token)	16
2.2 代码基本框架	16
2.3 预处理指令	17
2.3.1 <code>#include</code> 头文件包含	17
2.3.2 <code>#define</code> 宏定义	17
2.3.3 带参数宏定义的副作用	17
2.4 函数声明、实现与文件结构	18
2.4.1 头文件 (Header File) —— 后缀为 <code>.h</code>	18
2.4.2 源文件 (Source File) —— 后缀为 <code>.c</code> / <code>.cpp</code>	18
2.5 语句与表达式	18
2.6 控制流程	19
2.7 作用域 (Scope)	19
2.7.1 局部变量遮蔽 (Variable Shadowing)	19
2.8 递归 (Recursion)	20

目录	3
2.8.1 递归的两大核心要素	20
2.8.2 递归示例：阶乘计算	20
2.8.3 递归示例：斐波那契数列	20
2.8.4 递归与栈的关系	20
2.9 命令行参数 (argc, argv)	21
第三章 数据类型	22
3.1 四种基本数据类型	23
3.2 算术类型详解（大小与取值范围）	23
3.3 sizeof 运算符详解	24
3.4 类型转换 (Type Casting)	24
3.4.1 隐式类型转换 (Implicit Conversion)	24
3.4.2 显式强制类型转换 (Explicit Casting)	24
3.5 整型逆向溢出 (Overflow) 的边界现象	25
3.6 枚举类型 (Enum)	25
3.7 空类型 (Void)	25
3.8 派生类型 (Derived Types)	26
3.8.1 结构体类型 (struct)	26
3.8.2 结构体内存对齐 (Padding)	26
3.8.3 数组类型与指针类型	26
3.9 C99/C11 核心新特性	27
3.9.1 指定初始化器 (Designated Initializers)	27
3.9.2 柔性数组成员 (Flexible Array Member)	27
第四章 指针与地址	28
4.1 指针的本质	29
4.2 指针与数组的关系	29
4.3 指针与字符串	30
4.4 多级指针	30
4.5 动态内存管理	31
4.6 函数指针与回调	32
4.7 const 与指针的组合	33
4.8 复杂指针声明解析	33
4.8.1 指针数组 vs 数组指针	33
4.8.2 函数指针数组	34
4.9 实战：单链表的 C 语言实现	35
第五章 运算符与位运算	37
5.1 运算符分类	38
5.1.1 算术运算符	38
5.1.2 关系运算符	38
5.1.3 逻辑运算符	38
5.1.4 位运算符	38

目录	4
5.1.5 赋值运算符	38
5.1.6 杂项运算符	39
5.2 运算符优先级可视化	39
5.3 位运算高级技巧	40
5.3.1 异或 (XOR) 原地值交换算法 (无需临时变量)	40
5.3.2 判断一个数是否为 2 的幂次 (2^k)	40
5.3.3 常用位运算操作	40
5.4 原码、反码与补码	40
第六章 存储类别与内存模型	41
6.1 存储类别 (Storage Classes)	42
6.2 内存四区模型	42
6.3 static 关键字的”双重人格”	43
6.4 类型导出定义 (typedef)	43
6.5 位域 (Bit Fields)	43
6.6 共用体 (Union)	44
6.6.1 利用 union 检测系统大端序和小端序 (Endianness)	44
第七章 输入输出与文件	45
7.1 C 语言输入输出核心概念	46
7.2 stdio.h 的函数管理	46
7.2.1 格式化输出: printf	46
7.2.2 格式化输入: scanf	46
7.2.3 字符级 I/O	46
7.2.4 字符串级 I/O	46
7.2.5 现代 C 语言中 gets() 的重大安全隐患	47
7.3 文件操作	47
7.3.1 打开与关闭文件	47
7.3.2 字符级文件 I/O	47
7.4 二进制文件读写与块操作	47
7.4.1 特点与应用场景	48
7.5 完整文件操作示例	48
7.5.1 文本文件读写完整示例	48
7.5.2 二进制文件读写结构体完整示例	49
7.5.3 文件位置指针操作	49
7.6 输入缓冲区陷阱与解决方案	50
7.6.1 问题根源	50
7.6.2 标准解决方案	51
第八章 预处理器与工程实践	52
8.1 条件编译	53
8.1.1 #ifdef / #ifndef / #endif	53
8.1.2 #if 表达式编译	53

目录	5
8.2 头文件保护符 (Include Guard)	54
8.2.1 传统 <code>#ifndef</code> 保护模式	54
8.2.2 <code>#pragma once</code> 模式	54
8.3 多文件项目管理	55
8.4 Makefile 基础	55
8.5 常用编译选项	56
8.6 调试技巧	56
8.6.1 使用 GDB 调试	56
8.6.2 常见运行时错误排查	57
8.7 静态库与动态库	57
8.7.1 静态库 (Static Library)	57
8.7.2 动态库/共享库 (Shared Library)	57
8.8 从 Makefile 到 CMake	58
8.9 未定义行为 (UB) 常见清单	58
附录 A C 标准库函数速查手册 (I/O、内存与字符串)	59
A.1 C 标准库头文件总览	60
A.2 <code><stdio.h></code> 标准输入输出库	61
A.2.1 格式化输入输出	61
A.2.2 字符与字符串 I/O	61
A.2.3 文件操作	62
A.2.4 其他 I/O 与错误处理	62
A.3 <code><stdlib.h></code> 标准通用工具库	63
A.3.1 内存管理	63
A.3.2 数值转换	63
A.3.3 绝对值与除法	63
A.3.4 随机数与程序控制	64
A.3.5 查找与排序	64
A.3.6 多字节与宽字符转换	64
A.4 <code><string.h></code> 字符串处理库	65
A.4.1 字符串操作	65
A.4.2 内存操作	66
A.4.3 其他字符串操作	66
附录 B C 标准库函数速查手册 (数学、时间与字符)	67
B.1 <code><math.h></code> 数学函数库	68
B.1.1 常用数学函数	68
B.1.2 三角函数	68
B.1.3 双曲函数	69
B.1.4 其他数学函数	69
B.2 <code><ctype.h></code> 字符分类与转换库	70
B.3 <code><time.h></code> 时间与日期库	71
B.4 <code><complex.h></code> 复数运算库 (C99)	72

B.5	<tgmath.h> 泛型数学宏库 (C99)	72
B.6	<wchar.h> 宽字符 I/O 库 (C95)	73
B.7	<wctype.h> 宽字符分类库 (C95)	74
附录 C	C 标准库函数速查手册 (系统、类型与环境)	75
C.1	<assert.h> 断言库	76
C.2	<errno.h> 错误码库	76
C.3	<stdarg.h> 可变参数库	77
C.4	<limits.h> 与 <float.h> 极限值常量	78
C.4.1	limits.h 整型极限	78
C.4.2	float.h 浮点型极限	78
C.5	<stddef.h> 标准定义库	78
C.6	<setjmp.h> 非局部跳转库	79
C.7	<signal.h> 信号处理库	79
C.8	<locale.h> 本地化设置库	80
C.9	<stdbool.h> 布尔类型库 (C99)	80
C.10	<inttypes.h> 固定宽度整型库 (C99)	81
C.11	<stdint.h> 精确宽度整型库 (C99)	81
C.12	<fenv.h> 浮点环境库 (C99)	82
C.13	<iso646.h> 替代拼写宏库 (C95)	82

第一章 C 语言简介

本章是全书的起点。我们首先回答一个根本问题：**为什么要学习 C 语言**？然后沿着一个程序“从文本到运行”的完整生命周期，建立起对编译过程和程序结构的整体认识，并初步勾勒出 C 程序在内存中的布局，为后续的数据类型、指针、存储类别等章节埋下伏笔。

1.1 诞生背景与历史地位

C 语言诞生于 1972 年，由丹尼斯·里奇（Dennis Ritchie）在美国贝尔实验室（Bell Laboratories）设计。它的前身是肯·汤普逊（Ken Thompson）为 UNIX 系统编写的 B 语言，而 C 语言最初的核心目标非常明确：**用一种既能表达底层硬件操作、又具备良好可读性与可移植性的语言来重写 UNIX 操作系统**。

- **编程范式**：面向过程的（Procedure-Oriented）编程语言。
- **创始人**：丹尼斯·里奇（Dennis Ritchie）。
- **诞生地点**：美国贝尔实验室（Bell Laboratories）。
- **诞生时间**：1972 年。

C 语言的标准化历程如下：

- **K&R C**：1978 年《The C Programming Language》一书定义的非正式规范。
- **ANSI C / C89**：1989 年由美国国家标准学会（ANSI）制定，1990 年被 ISO 采纳为 C90，是经典的国际标准。
- **C99 / C11 / C17**：陆续加入变长数组、// 注释、inline、复数类型、多线程支持等特性。
- **GNU C / gcc**：GNU 计划在标准之外扩展的方言，配套编译器为 GCC（GNU Compiler Collection）。
- **MSVC**：微软的 Visual C++ 编译器，常用于 Windows 平台。

1.2 C 语言的特点

C 语言之所以能半个世纪长盛不衰，源于它一系列独特的设计取舍。可以用一句话概括它的哲学：**贴近硬件、信任程序员、只提供必要的抽象**。

- **简洁紧凑**：关键字少（C89 仅 32 个），语法规则精炼，没有内建的复杂类型系统。
- **贴近硬件**：支持指针、位运算和直接内存访问，可精确控制数据在机器上的表示与布局。
- **可移植**：标准不绑定具体硬件，同一份源码经重新编译即可在不同平台运行。
- **效率极高**：生成的机器码性能接近汇编，因此长期是操作系统、嵌入式与高性能计算的首选。
- **结构化**：以函数为基本单位，支持顺序、选择、循环三种基本控制结构。
- **生态庞大**：几乎所有主流语言（C++、Java、Python、Go、Rust 等）的运行时或解释器都用 C 或 C++ 实现。

1.3 为什么学习 C 语言

学习 C 语言有三重收益，它们层层递进：

1. **理解程序如何真正运行**：C 暴露了内存、地址、编译、链接等底层机制。掌握 C 之后，其他语言的“黑箱”会变得透明。
2. **获得系统编程能力**：操作系统内核、设备驱动、嵌入式固件、数据库引擎大量使用 C 编写，这是与硬件对话的通用语言。
3. **构建扎实的思维基础**：指针与内存管理训练逻辑严密性，这种思维迁移到任何编程语言都受用。

1.4 从源代码到可执行文件

初学者常把“写代码”与“运行程序”混为一谈，实际上从一份 .c 源文件到最终的可执行程序，中间要经过四个清晰的阶段。理解这条流水线，是理解 C 语言工程实践的关键。

1.4.1 四个阶段

1. **预处理 (Preprocessing)**：处理所有以 # 开头的指令。展开 #include 头文件、替换宏定义、展开条件编译、删除注释。生成预处理文件 .i。
2. **编译 (Compilation)**：将预处理后的文本翻译成汇编语言，并进行语法分析、语义检查与优化。生成汇编文件 .s。
3. **汇编 (Assembly)**：将汇编语言翻译成机器能够识别的二进制机器指令，生成目标文件 .o (Windows 上为 .obj)。
4. **链接 (Linking)**：将多个目标文件以及系统库文件合并、重定位，最终生成可在操作系统上运行的可执行文件 (.exe 或 .out)。

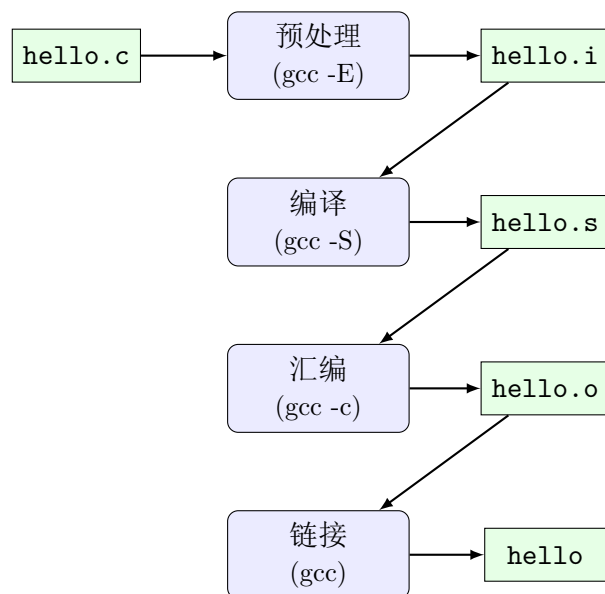


图 1.1: 从源代码到可执行文件的四个阶段

1.4.2 各阶段对照表

表 1.1: 编译四阶段：输入、输出与工具

阶段	输入	输出	gcc 选项	主要工作
预处理	.c	.i	-E	展开头文件、宏替换、去注释
编译	.i	.s	-S	语法/语义分析、优化、生成汇编
汇编	.s	.o	-c	将汇编翻译为机器指令（可重定位）
链接	.o	可执行文件	（无）	合并目标文件与库、重定位符号

1.4.3 gcc 命令行示例

GCC 默认只保留最终产物，中间文件会被自动删除。如果希望逐步观察，可以用下列选项“分步执行”：

```
gcc -E hello.c -o hello.i      # 1. 预处理，生成 hello.i
gcc -S hello.i -o hello.s      # 2. 编译，生成汇编 hello.s
gcc -c hello.s -o hello.o      # 3. 汇编，生成目标文件 hello.o
gcc hello.o -o hello           # 4. 链接，生成可执行文件 hello
./hello                        # 运行（Windows 上为 .\hello.exe）

# 也可以一步到位（等价于依次执行上面四步）：
gcc hello.c -o hello
```

Listing 1.1: 分步编译命令

```
gcc -Wall -Wextra hello.c -o hello # 打开常用告警，强烈建议
gcc -g hello.c -o hello             # 生成调试信息，供 gdb 使用
gcc -O2 hello.c -o hello            # 开启二级优化
```

Listing 1.2: 常用的调试与告警选项

1.5 一个完整的 Hello World

下面是最经典的程序。虽然只有 5 行代码，却几乎涵盖了 C 程序的所有结构要素。

```
#include <stdio.h>           // 1. 预处理指令：引入标准输入输出库

int main(void)               // 2. 主函数：程序的唯一入口
{                             // 函数体开始
    printf("Hello, World!\n"); // 3. 调用库函数，输出字符串
    return 0;                 // 4. 返回 0 表示程序正常结束
}
```

Listing 1.3: Hello World 完整程序

1.5.1 逐行剖析

表 1.2: Hello World 逐行解析

代码	含义
#include <stdio.h>	预处理指令。把标准输入输出头文件的内容在编译前插入此处，从而获得 printf 的函数声明。
int main(void)	定义一个返回 int 的函数 main，它是程序运行的入口；void 表示不接受参数。
{ ... }	函数体的花括号，界定语句的集合，函数体内部就是一组顺序执行的语句。
printf(...)	一条函数调用语句，末尾的分号表示语句结束；\n 是换行转义字符。
return 0;	从 main 返回整数 0，向操作系统报告“成功退出”。

关于 return 的返回值：main 的返回值会作为进程的退出状态码传递给操作系统。约定 0 表示成功，非 0 表示异常。省略 return 语句时，C99 起会隐式返回 0，但显式写出更清晰。

1.6 程序的基本结构

一个 C 程序在形式上由若干“翻译单元”组成，而每个单元内部则包含下列四种成分：

- 预处理指令：以 # 开头，如 #include、#define，在编译前由预处理器处理。
- 函数：程序的基本执行单位。main 是必需且唯一的入口，其余函数供其调用。
- 语句：函数体内的执行单元，以分号 ; 结尾。
- 注释：供人阅读的说明文字，编译器会忽略。包括块注释 /* ... */ 与行注释 // ...。

```
#include <stdio.h>           /* 预处理指令 */

#define PI 3.14159           // 宏定义（也是预处理指令）

double area(double r);       // 函数声明（原型）

int main(void)               // 主函数
{
    double a = area(2.0);    // 语句：调用函数
    printf("area = %f\n", a);
    return 0;                // 语句
}

double area(double r)        // 函数定义
{
    return PI * r * r;       // 语句：使用宏
}
```

Listing 1.4: 一个包含多种结构成分的程序

1.6.1 标识符与关键字

标识符是程序员为变量、函数、类型等起名字所用的符号。其命名规则为：

- 由字母、数字和下划线组成，且不能以数字开头。
- 区分大小写（count 与 Count 是两个不同的标识符）。
- 不能使用关键字，也不建议与标准库函数重名。
- 见名知义：如 max_value 优于 x1。

关键字是语言保留的、具有特殊含义的标识符，程序员不能将其用作普通名字。C89 共 32 个关键字：

表 1.3: C89 的 32 个关键字			
auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

C99 及后续标准又增加了 inline、restrict、_Bool、_Complex、_Generic 等关键字。

1.7 开发流程与常用工具

编写一个 C 程序通常遵循“编辑 → 编译 → 运行 → 调试”的循环，涉及三类核心工具：

- 编译器：如 GCC、Clang、MSVC，完成前面介绍的编译四阶段，把源码翻译为可执行文件。
- 调试器：如 GDB、LLDB、Visual Studio Debugger，用于设置断点、单步执行、检查变量与调用栈。
- 构建工具：如 make、CMake、Ninja，当工程包含成百上千个源文件时，自动管理编译顺序与依赖，避免重复编译。

1.7.1 最小的编译运行示例

```
int main(void)
{
    return 0;
}
```

Listing 1.5: main.c: 最小可运行程序

在命令行中执行：

```
gcc -Wall main.c -o main      # 编译；-Wall 打开告警
./main                        # 运行；无输出，退出码为 0
echo $?                       # 查看上一条命令的退出码（Linux 下）
```

Listing 1.6: 编译与运行

```
gcc -g main.c -o main        # 带调试信息编译
gdb ./main                   # 启动调试器
# (gdb) break main           # 在 main 处设置断点
# (gdb) run                   # 运行至断点
# (gdb) next                  # 单步执行
```

Listing 1.7: 使用 GDB 调试

1.8 C 程序的内存模型概览

一个正在运行的 C 进程，其逻辑地址空间会被划分为若干区域。理解这张“地图”，是后续学习指针、数组越界、动态内存与存储类别的必备基础。

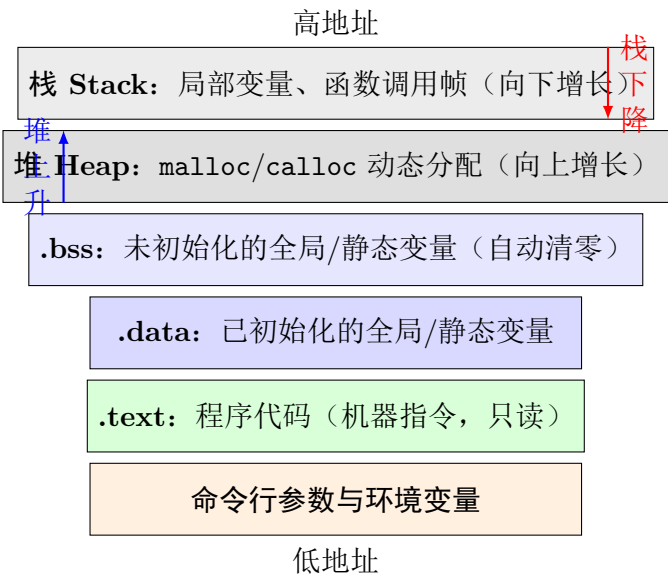


图 1.2: 典型 C 进程的地址空间布局

表 1.4: 各内存区域对比		
区域	内容	生命周期与特点
栈	局部变量、参数、返回地址	随函数调用自动分配与释放，速度快，空间有限
堆	malloc 等分配的内存	手动申请、手动 free，容量大但需自行管理
.bss	未初始化全局/静态变量	程序启动时置零，不占用可执行文件体积
.data	已初始化全局/静态变量	存储在可执行文件中，程序运行期间一直存在
.text	可执行机器指令	通常只读，防止程序被意外改写

为什么要先看这张图？因为它解释了 C 语言许多“为什么”：为什么局部变量在函数返回后值会失效（栈帧被回收）？为什么忘记 `free` 会造成内存泄漏（堆不会自动回收）？为什么修改字符串字面量会崩溃（位于只读段）？后续章节将逐一深入。

1.9 本章小结与常见误区

1.9.1 小结

- C 语言面向过程，1972 年诞生于贝尔实验室，以贴近硬件、可移植、高效著称。
- 源码到可执行文件要经过**预处理** → **编译** → **汇编** → **链接**四个阶段，产物依次为 `.i`、`.s`、`.o` 与可执行文件。
- 一个 C 程序由预处理指令、函数、语句和注释构成，`main` 是唯一入口。
- 标识符须遵循命名规则，且不得与 32 个关键字冲突。
- 运行时进程的地址空间分为栈、堆、`.bss`、`.data`、`.text` 等区域，是理解指针与内存管理的基础。

1.9.2 常见误区

- **忘记分号**：C 要求每条语句以 `;` 结尾。漏写分号往往报错在下一行，容易误导定位。
- **把 `=` 当成比较**：赋值 `=` 与相等比较 `==` 不同，混用会导致逻辑错误且编译器未必报错。
- **`main` 返回值随意**：应返回 `int`，成功用 0，异常用非零值；写成 `void main()` 不具可移植性。
- **混淆头文件与库**：头文件（`.h`）只提供**声明**，供编译阶段检查；真正的函数**实现**在库文件（`.a/.lib/.so/.dll`）中，由链接阶段引入。`#include` 并不会“引入实现”。
- **认为 `#include` 是“导入模块”**：它只是文本替换，把头文件内容原样插入到 `#include` 所在位置。
- **依赖未定义行为**：例如使用未初始化的变量、访问数组越界。这类代码可能“看起来能跑”，但结果不可预测。

第二章 程序结构与词法基础

2.1 词法单元 (Token)

C 语言程序由各种令牌 (Tokens) 组成，Token 是程序中具有独立语义的最小语法单位。

分类	定义	示例
关键字 (Keyword)	系统预留的、具有特定语法意义的单词	if, int, return, void
标识符 (Identifier)	用户自定义的各种名称，用于给变量、函数、数组等命名	a, b, number1
常量 (Constant)	在程序运行过程中，其值固定不变的量	3.1415926, 100
字面量 (Literal)	直接在源代码中写出的具体数值或文本	"ABC"
运算符 (Operator)	用于执行数学或逻辑运算的符号	+, -, *, /, =, --
分隔符 (Separator)	用于分隔各语法单元，辅助编译器解析代码结构	{, }, ;, (,), ,, ., ->

表 2.1: C 语言六大 Token 分类

命名规则：标识符只能由字母、数字和下划线组成，且第一个字符绝对不能是数字。同时，C 语言严格区分大小写（例如 Value 和 value 是两个完全不同的变量）。

底层机制：字面量是没有名字的“硬编码值”（通常直接硬编码在代码段中，无法直接对其取地址）；而常量通常是指用 const 修饰的变量，它有名字、内存类型和地址，只是在编译期被限制了修改权限。

2.2 代码基本框架

一个标准的 C 语言程序通常包含以下基本元素：

```
#include <stdio.h> // 预处理指令：包含标准输入输出库

int main() {       // 主函数：程序的执行入口
    int a;         // 变量声明
    printf("Hello");// 语句/表达式：调用打印函数
    return 0;      // 返回值：表示程序正常结束
}
```

Listing 2.1: 标准 C 语言程序框架

2.3 预处理指令

预处理指令在程序编译前由预处理器处理，以 # 开头。

2.3.1 #include 头文件包含

- `<stdio.h>`: 引用官方/标准库头文件（编译器会优先去系统标准库路径下寻找）。
- `"my.h"`: 引用用户自定义的本地头文件（编译器会优先在当前源文件所在的项目目录下寻找）。

2.3.2 #define 宏定义

- `PI 3.14`: 定义常量/字面量。
- `MAX(a, b) ((a) > (b) ? (a) : (b))`: 定义宏函数（类似于内联函数）。

2.3.3 带参数宏定义的副作用

宏定义的本质是纯文本替换，在使用时极易因为逻辑优先级或者多次求值引发错误。

```
// 陷阱 1: 未加括号导致优先级错误
#define MULTIPLY(a, b) a * b
// 期望 2 * (3 + 4) = 14, 实际替换为 2 * 3 + 4 = 10
int res = MULTIPLY(2, 3 + 4);

// 陷阱 2: 双重求值副作用
#define MAX(a, b) ((a) > (b) ? (a) : (b))
int x = 5, y = 3;
// 实际替换为: ((x++) > (y) ? (x++) : (y))
// 导致 x 被自增了两次, 最终结果和 x 的值都不符合预期!
int z = MAX(x++, y);
```

Listing 2.2: 宏定义陷阱示例

2.4 函数声明、实现与文件结构

C 语言通常将代码拆分为头文件和源文件来管理。

2.4.1 头文件 (Header File) ——后缀为 .h

- 作用：存储所有希望暴露给外部使用的函数声明、变量或接口 (Interface)。

- 示例内容：

1. `int Add(int a, int b);` ——函数声明 (提供给外部调用的接口)。
2. `int a = 5;` ——变量。
3. `const float a = 3.14;` ——常量。

2.4.2 源文件 (Source File) ——后缀为 .c / .cpp

- 作用：存储具体的业务逻辑与代码实现。
- 规则：在头文件中声明的函数，**必须在源文件中给出具体实现**。

```
int Add(int a, int b) {  
    return a + b;  
}
```

Listing 2.3: 源文件实现示例

2.5 语句与表达式

1. 语句 (Statement) ——代表”行为”

- 控制语句：如 `if (...) { ... }`
- 调用语句：如 `Add(1, 2);`
- 赋值语句：如 `a = b = c = 0;` (支持从右往左的连续赋值)。

2. 表达式 (Expression) ——代表”计算”

- 示例：`a + b;` (计算 `a` 加 `b` 的值，表达式本身具有返回值)。

2.6 控制流程

通过特定关键字控制程序执行顺序，主要分为以下三类：

1. 分支结构 / 选择循环

- `if (...) -- else if (...) -- else`: 若……则……或者……则……否则……
- `switch (...) { case 1: ... default: ... }`: 多分支选择语句

2. 循环语句 / 迭代流程

- `while (...)`: 当……时，执行循环
- `do { ... } while (...)`: 先执行一次体，再判断条件直到……
- `for (x; y; z)`: 最常用的标准循环/迭代语句

3. 跳转语句 / 分流作用

- `break`: 退出当前循环或 `switch` 结构
- `continue`: 跳过本次循环剩余代码，直接进入下一次循环
- `return`: 函数返回，结束当前函数的运行
- `goto`: 无条件跳转到指定标签（极易破坏代码结构，非特殊底层处理不建议使用）

2.7 作用域 (Scope)

作用域决定了变量的可见性和生命周期。

1. **命名唯一性**: 在同一个作用域内，变量名必须是唯一的。
2. **最小单元**: 大括号 `{ }` 是 C 语言中划分作用域的最小单元（块级作用域）。
3. **隔离规则**: 外层作用域不可以访问内层作用域中定义的变量（内层变量外部不可访问）。
4. **生命周期**: 一旦代码执行超过了变量所在的作用域，该变量就不能再被访问。
5. **继承与暴露**: 作用域具有继承性，内层作用域可以访问外层作用域的变量（从内向外层层暴露）。

2.7.1 局部变量遮蔽 (Variable Shadowing)

当内层作用域声明了一个与外层同名的变量时，外层变量在内层会暂时失效（被遮蔽）。这容易让人在多层循环或嵌套代码中写出逻辑 Bug:

```
int i = 100;
for (int i = 0; i < 5; i++) {
    // 这里的 i 访问的是循环变量 i (0~4)，外层的 i = 100 被遮蔽了
    printf("%d ", i);
}
printf("\n外层 i 的值依旧是: %d", i); // 此时输出 100
```

Listing 2.4: 变量遮蔽示例

2.8 递归 (Recursion)

递归是指函数在其函数体内直接或间接调用自身的编程技术。递归的本质是将一个大问题分解为规模更小的同类子问题，直到子问题规模缩小到可以直接求解的基准情形 (Base Case)。

2.8.1 递归的两大核心要素

- 基准情形 (Base Case):** 递归必须有一个明确的终止条件，否则将陷入无限递归导致栈溢出 (Stack Overflow)。
- 递推关系 (Recursive Step):** 每次递归调用必须使问题规模向基准情形收敛。

2.8.2 递归示例：阶乘计算

数学上 $n! = n \times (n - 1)!$ ，且 $0! = 1$ 。

```
int factorial(int n) {  
    if (n == 0) return 1;           // 基准情形  
    return n * factorial(n - 1);    // 递推关系  
}
```

Listing 2.5: 递归实现阶乘

2.8.3 递归示例：斐波那契数列

斐波那契数列定义为 $F(n) = F(n - 1) + F(n - 2)$ ，且 $F(0) = 0, F(1) = 1$ 。

```
int fibonacci(int n) {  
    if (n <= 1) return n;  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

Listing 2.6: 递归实现斐波那契

注意：上述朴素递归实现存在大量重复子问题计算，时间复杂度为 $O(2^n)$ 。工程实践中通常改用迭代法或记忆化搜索 (Memoization) 优化至 $O(n)$ 。

2.8.4 递归与栈的关系

每次函数调用都会在调用栈 (Call Stack) 上压入一个新的栈帧 (Stack Frame)，保存局部变量、参数和返回地址。递归深度过大时会耗尽栈空间，因此递归适用于问题规模可控的场景。对于树遍历、分治算法等天然具有递归结构的问题，递归是最自然的表达方式。

2.9 命令行参数 (argc, argv)

实际工程中的 C 程序通常通过命令行接收参数。主函数的标准签名支持接收命令行参数：

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("参数个数: %d\n", argc);
    for (int i = 0; i < argc; i++) {
        printf("argv[%d] = %s\n", i, argv[i]);
    }
    return 0;
}
```

Listing 2.7: 命令行参数示例

- **argc (Argument Count)**: 命令行参数的总个数，至少为 1（程序名本身算一个参数）。
- **argv (Argument Vector)**: 字符串数组，argv[0] 是程序名，argv[1] 到 argv[argc-1] 是用户传入的参数。

例如运行命令：

```
./program input.txt 100
```

则 argc = 3, argv[0] = "./program", argv[1] = "input.txt", argv[2] = "100"。

第三章 数据类型

3.1 四种基本数据类型

C 语言的数据类型主要分为以下四大类：

- 1. **算术类型（基础）**：包括整型（`int`）、字符型（`char`）、浮点型（`float`、`double`）等。
- 2. **枚举类型**：用于定义一组命名的整型常量。例如：`enum Sex { man = 1, woman = 0 };`。
- 3. **空类型 (Void)**：表示无类型或无返回值。
- 4. **派生类型**：由基本类型延伸出来的复杂类型，包括数组（如 `int a[10]`）、指针（如 `int *p`）、结构体（如 `struct S1`）等。

3.2 算术类型详解（大小与取值范围）

不同类型在内存中所占占用的字节（Byte）大小及对应的数值范围如下（以现代 32 位/64 位综合环境为例）：

类型	大小	取值范围
<code>char</code> （有符号字符型）	1 Byte	-128 ~ 127
<code>unsigned char</code> （无符号字符型）	1 Byte	0 ~ 255
<code>signed char</code> （显式有符号字符型）	1 Byte	-128 ~ 127
<code>short</code> （短整型）	2 Byte	-32768 ~ 32767
<code>int</code> （整型）	4 Byte	-2147483648 ~ 2147483647
<code>long</code> （长整型）	4/8 Byte	平台相关（Windows 为 4，Linux 64 位为 8）
<code>long long</code>	8 Byte	$-2^{63} \sim 2^{63} - 1$
<code>float</code> （单精度浮点）	4 Byte	约 $\pm 3.4 \times 10^{38}$ ，6-7 位有效数字
<code>double</code> （双精度浮点）	8 Byte	约 $\pm 1.8 \times 10^{308}$ ，15-16 位有效数字

表 3.1: C 语言基本数据类型大小与范围

3.3 sizeof 运算符详解

sizeof 是 C 语言的一元运算符（非函数），在编译期计算类型或变量所占的字节数，返回类型为 `size_t`（无符号整型）。

```
printf("int:      %zu bytes\n", sizeof(int));
printf("double:   %zu bytes\n", sizeof(double));

int arr[10];
printf("arr 总大小: %zu bytes\n", sizeof(arr));          // 40
printf("数组元素个数: %zu\n", sizeof(arr)/sizeof(arr[0])); // 10
```

Listing 3.1: sizeof 使用示例

关键注意事项：

- sizeof 在编译期求值，不产生运行时开销。
- 当数组作为参数传递给函数时，会退化为指针，此时 sizeof 返回的是指针大小而非数组总大小。
- 格式说明符应使用 %zu（C99 标准）来打印 size_t 类型。

3.4 类型转换（Type Casting）

3.4.1 隐式类型转换（Implicit Conversion）

编译器在运算过程中自动进行的类型转换，遵循“向高精度/大范围类型提升”的原则：

```
int a = 5;
double b = 2.5;
double result = a + b; // int 自动提升为 double, 结果为 7.5

char c = 'A';          // char 隐式提升为 int, 值为 65
int n = c + 1;          // n = 66
```

Listing 3.2: 隐式类型转换

隐式转换的优先级链（从低到高）：char/short → int → unsigned → long → double。

3.4.2 显式强制类型转换（Explicit Casting）

程序员通过强制转换语法 (type)expression 手动指定目标类型：

```
double pi = 3.14159;
int n = (int)pi;      // 截断小数部分, n = 3

int x = 7, y = 2;
double avg = (double)x / y; // 先将 x 转为 double, 再做除法, 结果为 3.5
// 若不加转换: x / y 为整数除法, 结果为 3
```

Listing 3.3: 显式类型转换

转换风险：

- 大范围类型转小范围类型时，高位数据会被截断丢失。
- 有符号转无符号时，负数会被解释为极大的正数（补码重新解释）。
- 浮点转整型时，小数部分直接截断（非四舍五入）。

3.5 整型逆向溢出（Overflow）的边界现象

计算机内部用补码存储数字。一旦数值超过了该类型能表示的最大边界，就会发生”溢出”，产生反常的现象：

```
char c = 127;
c = c + 1;
printf("%d", c); // 输出结果为 -128! 因为二进制补码 01111111 加 1 变成了 10000000 (即 -128 的补码)。
```

Listing 3.4: 整型溢出示例

3.6 枚举类型 (Enum)

1. 定义语法: `enum 类型名 { 字面量 = 值, 字面量 = 值, ... }`; 其本质上仍然是整数类型。
2. 代替方案: 可以用 `#define` 预处理命令来定义类似的常量组合。
3. 自增特性:
 - 若写 `enum A { A, B, C }`; → 默认 $A = 0, B = 1, C = 2$ 。
 - 若写 `enum A { A = 1, B, C }`; → 后面依次递增, $B = 2, C = 3$ 。
 - 若写 `enum A { A = 1, B = 3, C }`; → C 在 B 的基础上递增, $C = 4$ 。
4. 应用场景: 如果定义的枚举类型值是连续的, 那么它非常适合作为 `switch` 语句的判定条件, 增强代码可读性。

3.7 空类型 (Void)

- `void` (纯控制流程): 主要用于定义没有返回值的函数。
- `void*` (万能指针/泛型指针): 可以指向任何数据类型的指针。

```
#include <stdlib.h>
int *arr = (int*)malloc(5 * sizeof(int)); // 将 void* 强转为 int*
if (arr != NULL) {
    arr[0] = 10;
    free(arr); // 记得释放内存
}
```

Listing 3.5: `void*` 使用示例

3.8 派生类型 (Derived Types)

3.8.1 结构体类型 (struct)

```
struct Student {  
    char name[20];  
    int age;  
} s1; // 声明结构体的同时定义变量 s1
```

Listing 3.6: 结构体定义示例

注意：C 语言中没有类和对象的概念，绝不能写出像 Java/C# 那样的 `A a = new A();`。

3.8.2 结构体内存对齐 (Padding)

结构体在内存中的总大小，往往不等于各个成员大小的总和。为了提高 CPU 读取内存的效率，编译器会进行“内存对齐”：

```
struct Target {  
    char a;    // 1 字节  
    int b;     // 4 字节  
};  
// 很多人认为 sizeof(struct Target) 是 1 + 4 = 5 字节  
// 但在 32/64位系统下，实际输出是 8 字节！因为 char a 后面被填充 (Padding) 了 3 个空字节，以对齐 4 字节的 int。
```

Listing 3.7: 内存对齐示例

对齐规则：每个成员的起始地址必须是其自身大小的整数倍。结构体总大小必须是最大成员大小的整数倍。可通过调整成员声明顺序（将大类型成员放在前面）来减少填充浪费。

3.8.3 数组类型与指针类型

- 数组类型：示例：一维数组 `int a[10];`；或多维数组 `int a[3][4];`；
- 指针类型：示例：一级指针 `int *p;`；或二级/多级指针 `int **p;`；

3.9 C99/C11 核心新特性

3.9.1 指定初始化器 (Designated Initializers)

C99 引入了指定初始化器，允许在初始化结构体或数组时，通过成员名或数组下标显式指定赋值目标。这极大提高了代码的可读性和维护性，特别是在结构体成员较多时。

```
struct Point {
    int x;
    int y;
    int z;
};

// 传统初始化（必须按顺序）
struct Point p1 = {1, 2, 3};

// C99 指定初始化（可乱序，未指定成员默认为 0）
struct Point p2 = {.y = 2, .x = 1, .z = 3};

// 数组指定初始化
int arr[10] = {[0] = 10, [5] = 20, [9] = 30};
// 等价于 arr[0]=10, arr[5]=20, arr[9]=30, 其余为 0
```

Listing 3.8: 指定初始化器示例

3.9.2 柔性数组成员 (Flexible Array Member)

C99 允许在结构体的最后一个成员声明一个不指定大小的数组（通常写作 `type name[];`）。这常用于实现变长数据包或动态字符串。

```
struct Packet {
    int length;
    char data[]; // 柔性数组，不占用 sizeof 空间
};

// 分配内存时：sizeof(Packet) + 实际数据长度
int payload_size = 100;
struct Packet *pkt = malloc(sizeof(struct Packet) + payload_size);
pkt->length = payload_size;
// 现在可以安全访问 pkt->data[0] 到 pkt->data[99]
```

Listing 3.9: 柔性数组示例

注意：柔性数组成员必须是结构体的最后一个成员，且结构体必须至少包含一个其他成员。

第四章 指针与地址

4.1 指针的本质

指针是 C 语言最核心也最强大的特性。指针变量存储的是另一个变量的内存地址，而非数据本身。

```
int a = 10;
int *p = &a;    // p 存储的是 a 的内存地址
printf("%p\n", (void*)p);    // 打印地址
printf("%d\n", *p);          // 解引用：通过地址获取值，输出 10
```

Listing 4.1: 指针基础示例

- &（取地址符）：获取变量的内存地址。
- *（解引用符/间接寻址符）：通过指针访问其所指向地址中存储的值。
- 指针本身也是一个变量，它有自己的内存地址和所占空间（32 位系统为 4 字节，64 位系统为 8 字节）。

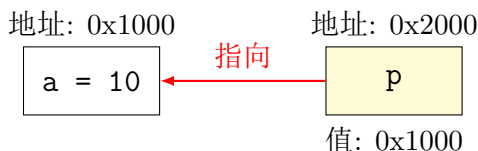


图 4.1: 指针变量 `p` 指向变量 `a` 的内存关系图

4.2 指针与数组的关系

在 C 语言中，数组名本质上就是指向数组首元素的常量指针。

```
int arr[5] = {1, 2, 3, 4, 5};
int *p = arr;    // 等价于 int *p = &arr[0];
printf("%d", *(p + 2)); // 输出 3, 等价于 arr[2]
```

Listing 4.2: 指针与数组

指针算术运算：指针加减整数时，实际移动的字节数 = 整数值 × sizeof(指针指向的类型)。

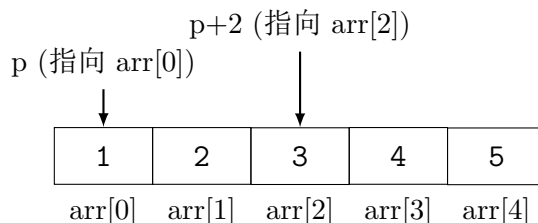


图 4.2: 指针算术运算与数组内存布局

4.3 指针与字符串

C 语言中的字符串是以 `\0` 结尾的字符数组。字符串字面量在内存中以只读方式存储，返回指向首字符的指针。

```
char *str = "Hello";    // str 指向字符串字面量的首地址
printf("%c\n", *str);   // 输出 'H'
printf("%c\n", *(str+1)); // 输出 'e'

// 注意：不可修改字符串字面量
// str[0] = 'h';        // 未定义行为，可能导致段错误
```

Listing 4.3: 指针与字符串

4.4 多级指针

指针可以指向另一个指针，形成多级间接寻址。二级指针常用于修改指针本身的值（如动态分配二维数组、修改函数外部的指针变量）。

```
int a = 10;
int *p = &a;
int **pp = &p;

printf("%d\n", *p);    // 输出 10，一次解引用
printf("%d\n", **pp);  // 输出 10，两次解引用

**pp = 20;             // 通过二级指针修改 a 的值
printf("%d\n", a);     // 输出 20
```

Listing 4.4: 二级指针示例

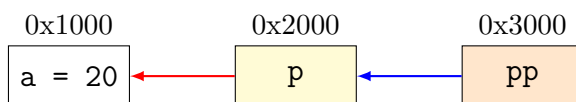


图 4.3: 二级指针链式指向关系

4.5 动态内存管理

C 语言通过标准库 `<stdlib.h>` 提供在堆（Heap）上动态分配内存的能力。

- `malloc(size_t size)`: 在堆上分配指定字节数的内存，返回 `void*`，内容未初始化（垃圾值）。
- `calloc(size_t n, size_t size)`: 分配 n 个大小为 `size` 的连续内存块，并自动初始化为 0。
- `realloc(void* ptr, size_t size)`: 重新调整已分配内存的大小。若新大小更大，原有数据保留，新增部分未初始化。
- `free(void* ptr)`: 释放已分配的内存，防止内存泄漏。

```
#include <stdlib.h>

int *arr = (int*)malloc(10 * sizeof(int));
if (arr == NULL) {
    // 内存分配失败处理
    return -1;
}

// 使用 arr...
arr[0] = 42;

// 使用完毕后必须释放
free(arr);
arr = NULL; // 避免悬垂指针 (Dangling Pointer)
```

Listing 4.5: 动态内存管理示例

关键注意事项:

- 每次 `malloc` 必须配对一次 `free`，否则造成内存泄漏。
- `free` 之后的指针应立即置为 `NULL`，避免悬垂指针被误用。
- 不可对同一块内存 `free` 两次（Double Free），会导致未定义行为。

4.6 函数指针与回调

函数指针允许将函数作为参数传递，实现回调机制和多态行为。

```
int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

// 声明函数指针
int (*func_ptr)(int, int);

func_ptr = add;
printf("%d\n", func_ptr(3, 4)); // 输出 7

func_ptr = sub;
printf("%d\n", func_ptr(3, 4)); // 输出 -1
```

Listing 4.6: 函数指针示例

```
// 通用排序比较函数原型
typedef int (*CompareFunc)(const void*, const void*);

void mySort(int arr[], int n, CompareFunc cmp) {
    // 排序过程中调用 cmp 比较元素
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (cmp(&arr[j], &arr[j+1]) > 0) {
                int tmp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = tmp;
            }
        }
    }
}
```

Listing 4.7: 回调函数示例

4.7 const 与指针的组合

const 修饰指针时有三种经典组合方式，含义各不相同：

```
int a = 10, b = 20;

// 1. 指针常量：指针本身不可变，但指向的值可变
int *const p1 = &a;
*p1 = 30;      // OK
// p1 = &b;    // Error: p1 是常量

// 2. 常量指针：指向的值不可变，但指针本身可变
const int *p2 = &a;
// *p2 = 30;   // Error: *p2 是常量
p2 = &b;        // OK

// 3. 指向常量的常量指针：两者均不可变
const int *const p3 = &a;
// *p3 = 30;   // Error
// p3 = &b;     // Error
```

Listing 4.8: const 与指针的三种组合

记忆口诀：const 在 * 左侧 → 值不可变；const 在 * 右侧 → 指针不可变。

4.8 复杂指针声明解析

在 C 语言中，指针与数组、函数的组合会产生复杂的声明。理解它们的关键是 ** 从变量名开始，按照优先级向外结合 **。

4.8.1 指针数组 vs 数组指针

```
int *p1[10]; // 指针数组：数组，包含 10 个 int* 指针
int (*p2)[10]; // 数组指针：指针，指向一个包含 10 个 int 的数组

// 解析技巧：
// [] 优先级高于 *，所以 p1 先和 [] 结合，是数组。
// () 强制优先级，所以 p2 先和 * 结合，是指针。
```

Listing 4.9: 指针数组与数组指针

4.8.2 函数指针数组

```
int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

// 声明一个包含 2 个函数指针的数组
int (*ops[2])(int, int) = {add, sub};

printf("%d\n", ops[0](3, 4)); // 输出 7
printf("%d\n", ops[1](3, 4)); // 输出 -1
```

Listing 4.10: 函数指针数组示例

4.9 实战：单链表的 C 语言实现

作为指针和结构体的综合练习，以下实现了一个带头结点的单链表，包含插入、删除和遍历操作。这是衔接《数据结构》课程的经典案例。

```

#include <stdio.h>
#include <stdlib.h>

// 节点定义
typedef struct Node {
    int data;
    struct Node *next;
} Node;

// 初始化带头结点的链表
Node* initList() {
    Node *head = (Node*)malloc(sizeof(Node));
    head->next = NULL;
    return head;
}

// 头插法
void insertHead(Node *head, int val) {
    Node *newNode = (Node*)malloc(sizeof(Node));
    newNode->data = val;
    newNode->next = head->next;
    head->next = newNode;
}

// 删除第一个值为 val 的节点
void deleteVal(Node *head, int val) {
    Node *prev = head;
    Node *curr = head->next;
    while (curr) {
        if (curr->data == val) {
            prev->next = curr->next;
            free(curr);
            return;
        }
        prev = curr;
        curr = curr->next;
    }
}

// 遍历打印
void printList(Node *head) {
    Node *curr = head->next;
    while (curr) {
        printf("%d -> ", curr->data);
        curr = curr->next;
    }
    printf("NULL\n");
}

// 释放整个链表
void freeList(Node *head) {
    Node *curr = head;
    while (curr) {
        Node *tmp = curr;
        curr = curr->next;
        free(tmp);
    }
}

```

Listing 4.11: 单链表完整实现

第五章 运算符与位运算

5.1 运算符分类

C 语言提供了丰富的运算符，主要分为以下六大类：

5.1.1 算术运算符

`+`, `-`, `*`, `/`, `%` (取余), `++` (自增), `--` (自减)。

5.1.2 关系运算符

`==`, `!=`, `>`, `>=`, `<`, `<=`。

5.1.3 逻辑运算符

`&&` (与), `||` (或), `!` (非)。

注：逻辑运算符的“短路求值”特性：在逻辑表达式中，一旦前面的部分已经能够决定整个表达式的结果，编译器就会停止执行后面的表达式。

```
int a = 0, b = 5;
// 因为 a == 0 为假，逻辑与 && 的最终结果注定为假，所以后面的 ++b 根本不会被执行！
if (a && (++b > 0)) { /* 不执行 */ }
printf("%d", b); // 输出仍为 5，而不是 6！
```

Listing 5.1: 短路求值示例

5.1.4 位运算符

- `&` (按位与)：有 0 则 0，全 1 才 1。
- `|` (按位或)：有 1 则 1，全 0 才 0。
- `^` (按位异或)：不同为 1，相同为 0。
- `~` (按位取反)。
- `<<` (左移)、`>>` (右移)。

移位运算的数学本质：在不溢出的前提下，一个无符号整数左移 n 位 `<< n` 相当于该数 $\times 2^n$ ；右移 n 位 `>> n` 相当于该数 $\div 2^n$ (向下取整)。这比直接用乘除法指令执行速度更快。

5.1.5 赋值运算符

`=`, `+=`, `-=` 等 (包含存取/复合赋值)。

5.1.6 杂项运算符

- sizeof(): 返回类型或变量所占用的字节数 (Byte)。
- &: 取地址符 (获取变量的内存地址)。
- *: 解引用符/间接寻址符 (通过地址获取值)。
- ? : : 三目运算符 (条件表达式, 等价于 if-else 简写)。
- (), [], ., ->: 括号、数组下标及结构体成员访问符。

5.2 运算符优先级可视化

从高到低的优先级顺序如下表所示。同一行内的运算符优先级相同, 结合性决定求值方向。

1 (最高)	() [] -> .	← 左结合
2	++ -- ! ~ sizeof & * (单目/一元)	← 右结合
3	* / % (乘、除、取余)	
4	+ - (加、减)	
5	<< >> (移位)	
6	< > <= >= (关系比较: 大小)	
7	== != (关系比较: 相等)	
8	& (按位与)	← 左结合
9	^ (按位异或)	
10	(按位或)	
11	&& (逻辑与)	
12	(逻辑或)	
13	? : (三目/条件)	← 右结合
14 (最低)	= += -= ... , (赋值与逗号)	← 右结合

图 5.1: C 语言运算符优先级全图 (从上到下优先级递减)

工程建议: 当表达式涉及多种运算符混合时, 不要依赖记忆优先级, 而是显式使用括号来明确意图。

5.3 位运算高级技巧

5.3.1 异或（XOR）原地值交换算法（无需临时变量）

利用 $A \oplus B$ 相同为 0、不同为 1 以及自反的特性，可以不借助临时变量交换两个整数：

```
// 假设 int a = 60 (0011 1100), b = 13 (0000 1101)
a = a ^ b; // 得到中间值 0011 0001
b = a ^ b; // 此时 b 变为了原 a 的值 (0011 1100)
a = a ^ b; // 此时 a 变为了原 b 的值 (0000 1101)
```

Listing 5.2: XOR 交换算法

5.3.2 判断一个数是否为 2 的幂次 (2^k)

利用 $n \& (n - 1)$ 能够完美消除二进制中最低位 1 的数学特性：

```
// 若成立，说明 n 的二进制表示中仅含有一个 1，即它是 2 的整数次幂
if ((n > 0) && (n & (n - 1)) == 0) { /* 是 2 的幂次 */ }
```

Listing 5.3: 2 的幂次判定

5.3.3 常用位运算操作

- 置位 (Set Bit): $n \mid= (1 \ll k)$ ，将第 k 位置为 1。
- 清零 (Clear Bit): $n \&= \sim(1 \ll k)$ ，将第 k 位置为 0。
- 取反位 (Toggle Bit): $n \hat{=} (1 \ll k)$ ，翻转第 k 位。
- 检测位 (Test Bit): $(n \gg k) \& 1$ ，获取第 k 位的值。

5.4 原码、反码与补码

计算机内部一律使用补码来存储和计算整数：

- 原码：最高位为符号位（0 表示正，1 表示负），其余位表示数值绝对值。
- 反码：正数的反码与原码相同；负数的反码为原码除符号位外，其余各位按位取反（0 变 1，1 变 0）。
- 补码：正数的补码与原码相同；负数的补码为反码 + 1。

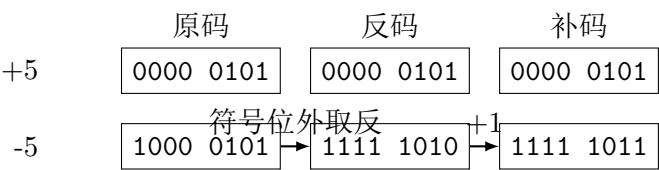


图 5.2: 原码、反码、补码转换关系（以 8 位为例）

第六章 存储类别与内存模型

6.1 存储类别 (Storage Classes)

- **auto** (自动): 默认类别。变量进入作用域时创建, 离开时自动销毁。
- **register** (寄存器): 建议编译器将变量放入 CPU 寄存器以极大提升读取速度。限制为一个字长 (如 4 字节), 且无法对寄存器变量使用 **&** 取地址符。
- **static** (静态): 具有隐藏和保持数据的作用。
- **extern** (外部声明): 用于告诉编译器, 该变量或函数的定义在别的文件中。

6.2 内存四区模型

C 程序运行时, 操作系统将进程的虚拟地址空间划分为四个主要区域, 每个区域承担不同的存储职责。

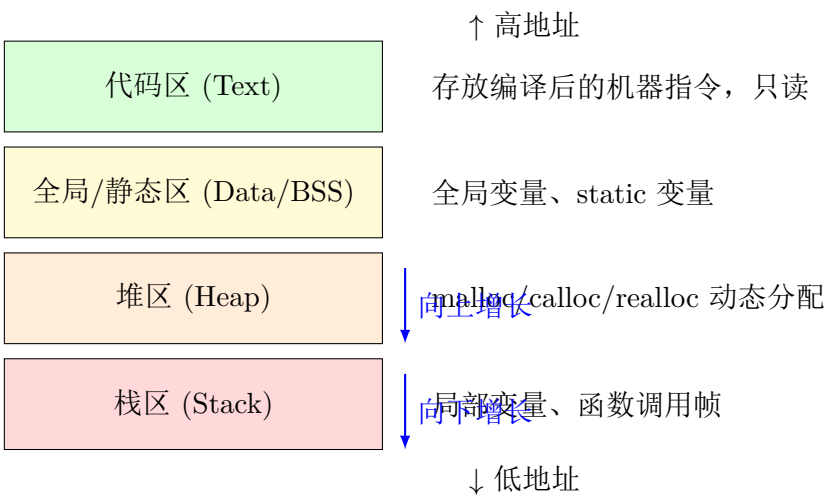


图 6.1: C 程序运行时内存四区布局

1. **代码区 (Text Segment):** 存放编译后的机器指令。该区域在物理上被标记为只读, 任何尝试修改代码区内容的操作都会触发段错误 (Segmentation Fault)。
2. **全局/静态区 (Data/BSS Segment):**
 - **Data 段:** 存放已初始化的全局变量和静态变量。
 - **BSS 段:** 存放未初始化或初始化为 0 的全局变量和静态变量, 程序加载时由系统自动清零。
3. **堆区 (Heap):** 由程序员通过 **malloc**、**calloc**、**realloc** 手动申请和 **free** 手动释放。堆区向高地址方向增长。若只申请不释放, 将导致内存泄漏。
4. **栈区 (Stack):** 由编译器自动管理。函数调用时在栈上压入栈帧 (Stack Frame), 包含局部变量、参数和返回地址。函数返回时栈帧自动弹出。栈区向低地址方向增长。栈空间有限 (通常几 MB), 递归过深会导致栈溢出。

6.3 static 关键字的”双重人格”

static 在 C 语言中修饰局部变量和修饰全局变量/函数时的作用完全不同：

1. **修饰局部变量（延长生命周期）**：函数调用结束后，该变量不会被销毁，其值会保留供下一次调用使用。它被存储在静态存储区（而非栈区），只在初次运行时初始化一次。
2. **修饰全局变量/函数（隐藏可见性）**：限制该全局变量或函数只能在当前 .c 文件内部被访问，其他文件即使使用 extern 也无法引用它，从而避免了跨文件命名的冲突。

```
void counter() {
    static int count = 0; // 仅首次调用时初始化为 0
    count++;
    printf("%d ", count);
}

int main() {
    counter(); // 输出 1
    counter(); // 输出 2
    counter(); // 输出 3
    return 0;
}
```

Listing 6.1: static 修饰局部变量

6.4 类型导出定义 (typedef)

- 作用：为已有数据类型定义一个新的易读别名。

```
typedef unsigned char byte; // 将 unsigned char 取别名为 byte
byte b1, b2;                // 声明变量，等价于 unsigned char b1, b2;
```

Listing 6.2: typedef 使用示例

6.5 位域 (Bit Fields)

允许在结构体中以”位（Bit）”为单位分配内存，常用于底层寄存器配置或网络协议解析：

```
struct Register {
    unsigned int low_bit : 1; // 仅占 1 位
    unsigned int mid_bits : 3; // 占用 3 位
};
```

Listing 6.3: 位域示例

6.6 共用体 (Union)

所有成员共享同一块内存，总大小等于其最大成员的大小。

6.6.1 利用 union 检测系统大端序和小端序 (Endianness)

大端序指高位字节存放在低地址，小端序指低位字节存放在低地址。我们可以用一段精简的共用体代码轻松检测当前计算机架构：

```
#include <stdio.h>
union EndianCheck {
    int i;
    char c;
};

int main() {
    union EndianCheck ec;
    ec.i = 1; // 十六进制为 0x00000001

    // 如果 c == 1, 说明低地址存储的是低位数据 (0x01), 为小端序 (大部分现代 PC 架构)
    if (ec.c == 1) {
        printf("当前系统为: 小端序 (Little Endian)\n");
    } else {
        printf("当前系统为: 大端序 (Big Endian)\n");
    }
    return 0;
}
```

Listing 6.4: 端序检测示例



图 6.2: 大端序与小端序的内存字节排列对比

第七章 输入输出与文件

7.1 C 语言输入输出核心概念

- 所有的设备都是文件（标准文件）：`stdin`（键盘）、`stdout`（屏幕）、`stderr`（标准错误输出）。
- 封装形式：以上文件及 I/O 函数都统一封装在标准库 `<stdio.h>` 中。

7.2 `stdio.h` 的函数管理

7.2.1 格式化输出：`printf`

- 原型：`printf(const char* format, ...);`
- 作用：格式化输出字符到屏幕。

7.2.2 格式化输入：`scanf`

- 原型：`scanf(const char* format, ...);`
- 作用：从屏幕按某个格式获取一个值。
- 参数说明：
 - `format`：是一个格式字符串，用于指定输出/输入的格式。
 - `...`：是一个可变长的参数列表，对应每一个前方的格式说明符的数据。
 - 格式说明符：如 `%d`（整型）、`%ld`（长整型）、`%.3f` 等，代表这个占位的区域存放什么类型的数据。
 - 转义字符：一个既定的组合，表示一个特殊的意义。例如：`\n`（换行）、`\r`（回车）、`\t`（制表符）、`\b`（退格）。

7.2.3 字符级 I/O

- `int getchar()`：从屏幕读取下一个可用的字符（从当前字符集）。
- `int putchar(int c)`：输出到屏幕某一个字符（从当前字符集）。

7.2.4 字符串级 I/O

- `char* fgets(char* str, int n, FILE* stream);`：从某个标准流中读取一些字符串。`n` 是从流中读取字符的个数。
- `int fputs(const char* str, FILE* stream);`：朝某个标准流注入字符串。`str` 是一个用于存放字符串的数组；`stream` 是一个标准流的指针。

7.2.5 现代 C 语言中 gets() 的重大安全隐患

老课本常提到的 gets(str) 因为无法限制读取输入的长度，极易导致缓冲区溢出（Buffer Overflow）甚至导致系统崩溃或被黑客攻击。在现代 C 标准（C11 及以后）中已经被完全废弃。

标准替代方案：请一律使用 fgets 代替。

```
char buf[100];  
// 最多只读取 99 个字符，并在末尾自动补 '\0'，绝对不会发生内存越界溢出  
fgets(buf, sizeof(buf), stdin);
```

Listing 7.1: 安全的字符串读取

7.3 文件操作

7.3.1 打开与关闭文件

- `FILE* fopen(const char* filename, const char* mode);`: 以流形式的某种模式打开文件。
 - filename: 是一个文件路径。
 - mode: 是一个访问模式符号，如 r/w/a（读/写/添），加上 + 表示拓展写模式。如果在后面追加 "b"（如 "rb", "wb"）则代表以二进制流模式访问。
- `int fclose(FILE* fp);`: 关闭文件。正常关闭返回 0，发生错误返回 EOF (-1)。

注：忘记 fclose 导致的资源泄漏：每次 fopen 打开文件都会占用操作系统的文件描述符及内存缓冲区。如果写程序忘记执行 fclose，会导致系统资源耗尽。

7.3.2 字符级文件 I/O

- `int fputc(int c, FILE* fp);`: 向指定文件流中写入一个字符。
- `int fgetc(FILE* fp);`: 从指定文件流中读取一个字符。

7.4 二进制文件读写与块操作

访问二进制文件时，访问模式必须添加一个 b，如 rb / wb / ab / rb+ / wb+ / ab+。

- `size_t fread(void* ptr, size_t size, size_t count, FILE* stream);`
- `size_t fwrite(const void* ptr, size_t size, size_t count, FILE* stream);`

7.4.1 特点与应用场景

- 特点：1. 读写快；2. 无精度损失。
- 适用场景：结构体、数组、图像、音频、视频。
- 高级特性：具备一级加密的效果（由于非文本格式，无法直接通过常规文本编辑器直观查看，会显示为乱码）。

7.5 完整文件操作示例

7.5.1 文本文件读写完整示例

```
#include <stdio.h>

int main() {
    // 写入文件
    FILE *fp = fopen("data.txt", "w");
    if (fp == NULL) {
        printf("打开文件失败\n");
        return 1;
    }
    fprintf(fp, "姓名: %s\n", "张三");
    fprintf(fp, "成绩: %d\n", 95);
    fclose(fp);

    // 读取文件
    fp = fopen("data.txt", "r");
    if (fp == NULL) {
        printf("打开文件失败\n");
        return 1;
    }
    char buf[256];
    while (fgets(buf, sizeof(buf), fp) != NULL) {
        printf("%s", buf);
    }
    fclose(fp);
    return 0;
}
```

Listing 7.2: 文本文件写入与读取

7.5.2 二进制文件读写结构体完整示例

```
#include <stdio.h>

struct Student {
    char name[20];
    int age;
    float score;
};

int main() {
    struct Student students[3] = {
        {"Alice", 20, 95.5},
        {"Bob", 21, 88.0},
        {"Charlie", 19, 92.3}
    };

    // 写入二进制文件
    FILE *fp = fopen("students.dat", "wb");
    if (fp == NULL) return 1;
    fwrite(students, sizeof(struct Student), 3, fp);
    fclose(fp);

    // 读取二进制文件
    struct Student buffer[3];
    fp = fopen("students.dat", "rb");
    if (fp == NULL) return 1;
    fread(buffer, sizeof(struct Student), 3, fp);
    fclose(fp);

    for (int i = 0; i < 3; i++) {
        printf("姓名: %s, 年龄: %d, 成绩: %.1f\n",
            buffer[i].name, buffer[i].age, buffer[i].score);
    }
    return 0;
}
```

Listing 7.3: 二进制文件读写结构体数组

7.5.3 文件位置指针操作

- `long ftell(FILE* fp)`: 返回当前文件位置指针相对于文件开头的偏移量（字节数）。
- `int fseek(FILE* fp, long offset, int whence)`: 将文件位置指针移动到指定位置。`whence` 可取 `SEEK_SET`（文件开头）、`SEEK_CUR`（当前位置）、`SEEK_END`（文件末尾）。
- `void rewind(FILE* fp)`: 将文件位置指针重置到文件开头，等价于 `fseek(fp, 0, SEEK_SET)`。

```
FILE *fp = fopen("data.bin", "rb");
// 跳过前 2 个结构体，直接读取第 3 个
```

```
fseek(fp, 2 * sizeof(struct Student), SEEK_SET);
struct Student third;
fread(&third, sizeof(struct Student), 1, fp);
fclose(fp);
```

Listing 7.4: fseek 随机访问示例

7.6 输入缓冲区陷阱与解决方案

初学者最常遇到的 I/O Bug 是：在使用 `scanf` 读取数字后，紧接着使用 `fgets` 读取字符串，结果 `fgets` 直接跳过，读取到一个空行。

7.6.1 问题根源

`scanf` 在读取数字时，遇到非数字字符（如换行符 `\n`）会停止读取，但会将该换行符留在输入缓冲区中。随后的 `fgets` 会立即读取到这个残留的换行符，认为已经读取了一行，从而跳过用户输入。

```
int age;
char name[50];

printf("请输入年龄：");
scanf("%d", &age); // 用户输入 "20<回车>", scanf 读取 20, 留下 \n

printf("请输入姓名：");
fgets(name, sizeof(name), stdin);
// fgets 立即读到残留的 \n, 直接返回, 导致 name 为空字符串!

printf("年龄： %d, 姓名： %s\n", age, name);
```

Listing 7.5: 输入缓冲区陷阱演示

7.6.2 标准解决方案

1. 清空输入缓冲区：在 `scanf` 后手动消耗掉残留字符，直到遇到换行符或 EOF。
2. 统一使用 `fgets`：放弃 `scanf`，全部使用 `fgets` 读取整行，再用 `sscanf` 或 `strtol` 解析。这是最安全、最推荐的做法。

```
// 方法 1：手动清空缓冲区
int c;
while ((c = getchar()) != '\n' && c != EOF);

// 方法 2（推荐）：统一使用 fgets + sscanf
char buf[100];
int age;

printf("请输入年龄：");
fgets(buf, sizeof(buf), stdin);
sscanf(buf, "%d", &age); // 从缓冲区解析数字，安全且不会残留

printf("请输入姓名：");
fgets(name, sizeof(name), stdin); // 正常等待用户输入
```

Listing 7.6: 清空缓冲区与统一 fgets 方案

第八章 预处理器与工程实践

8.1 条件编译

条件编译允许根据特定条件决定是否编译某段代码，是实现跨平台兼容和调试开关的核心工具。

8.1.1 #ifdef / #ifndef / #endif

```
#define DEBUG 1

#ifdef DEBUG
    printf("调试模式: 变量 x = %d\n", x);
#endif

#ifndef MAX_SIZE
    #define MAX_SIZE 100
#endif
```

Listing 8.1: 条件编译基本用法

- `#ifdef MACRO`: 如果 `MACRO` 已被 `#define` 定义，则编译其后代码。
- `#ifndef MACRO`: 如果 `MACRO` 未被定义，则编译其后代码。
- `#endif`: 结束条件编译块。
- `#else / #elif`: 可配合使用，形成多分支条件编译。

8.1.2 #if 表达式编译

```
#define VERSION 2

#if VERSION >= 3
    printf("版本 3 及以上功能\n");
#elif VERSION == 2
    printf("版本 2 功能\n");
#else
    printf("旧版本兼容代码\n");
#endif
```

Listing 8.2: if 表达式条件编译

8.2 头文件保护符（Include Guard）

在多文件项目中，同一头文件可能被多个源文件间接多次包含，导致重复声明错误。头文件保护符通过条件编译确保头文件内容仅被编译一次。

8.2.1 传统 `#ifndef` 保护模式

```
// student.h
#ifndef STUDENT_H
#define STUDENT_H

struct Student {
    char name[20];
    int age;
};

void printStudent(struct Student s);

#endif // STUDENT_H
```

Listing 8.3: 头文件保护符

命名约定：将头文件名转为大写，将点号替换为下划线，前后加下划线。例如 `my_header.h` 转为 `MY_HEADER_H`。

8.2.2 `#pragma once` 模式

```
#pragma once

struct Student {
    char name[20];
    int age;
};
```

Listing 8.4: `pragma once`

`#pragma once` 是编译器扩展指令，现代编译器（GCC、Clang、MSVC）均支持。它由编译器在文件级别保证唯一包含，书写更简洁，但不属于 C 标准。在存在符号链接或同一文件多条路径的工程中，`#ifndef` 模式更为可靠。

8.3 多文件项目管理

一个规范的 C 语言项目通常按以下结构组织：

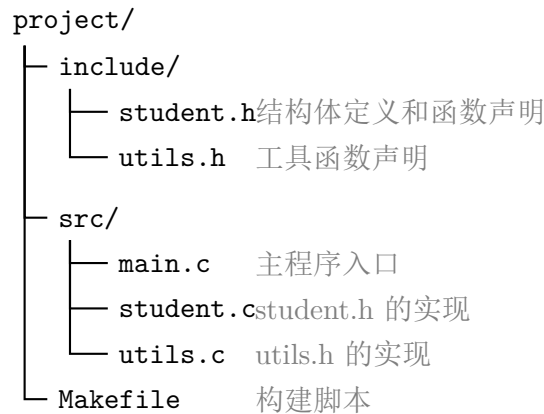


图 8.1: 典型项目文件结构

编译与链接流程：

1. 每个 .c 文件独立编译为 .o 目标文件。
2. 链接器将所有 .o 文件和系统库合并为最终可执行文件。
3. 修改某个 .c 文件时，只需重新编译该文件，无需重新编译整个项目。

8.4 Makefile 基础

Makefile 是 Unix/Linux 系统下的自动化构建工具，用于管理多文件项目的编译流程。

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c11
TARGET = program
SRCS = src/main.c src/student.c src/utils.c
OBJS = $(SRCS:.c=.o)

$(TARGET): $(OBJS)
    $(CC) $(CFLAGS) -o $(TARGET) $(OBJS)

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(OBJS) $(TARGET)
```

Listing 8.5: 简单 Makefile 示例

- CC: 指定编译器。
- CFLAGS: 编译选项，-Wall 开启所有警告，-Wextra 开启额外警告，-std=c11 指定 C11 标准。

- make: 执行构建。
- make clean: 清理编译产物。

8.5 常用编译选项

选项	作用
-Wall	开启所有常用警告
-Wextra	开启额外警告
-Werror	将警告视为错误，阻止编译通过
-g	生成调试信息（供 GDB 使用）
-O0 / -O1 / -O2 / -O3	优化等级（0 为不优化，3 为最高优化）
-std=c99 / c11 / c17	指定 C 语言标准版本
-o output	指定输出文件名
-c	仅编译不链接，生成.o 文件
-I dir	添加头文件搜索路径
-l lib	链接指定库（如 -lm 链接数学库）

表 8.1: GCC 常用编译选项

8.6 调试技巧

8.6.1 使用 GDB 调试

```
gcc -g -o program main.c // 编译时加 -g 生成调试信息
gdb ./program // 启动 GDB

(gdb) break main // 在 main 函数处设置断点
(gdb) run // 运行程序
(gdb) next // 单步执行（不进入函数）
(gdb) step // 单步执行（进入函数）
(gdb) print x // 打印变量 x 的值
(gdb) continue // 继续执行到下一个断点
(gdb) quit // 退出 GDB
```

Listing 8.6: GDB 基本命令

8.6.2 常见运行时错误排查

- **段错误 (Segmentation Fault):** 访问了非法内存地址，常见原因包括空指针解引用、数组越界、访问已释放的内存。
- **内存泄漏 (Memory Leak):** malloc 分配的内存未调用 free 释放。可使用 Valgrind 工具检测。
- **未定义行为 (Undefined Behavior):** 如 signed integer overflow、use-after-free 等，编译器不保证任何结果，可能导致难以复现的随机 Bug。

8.7 静态库与动态库

在实际工程中，通常将常用的功能打包成库 (Library)，供多个项目复用。C 语言支持两种库：静态库和动态库。

8.7.1 静态库 (Static Library)

静态库在编译时直接将目标代码复制到最终的可执行文件中。生成的可执行文件体积较大，但运行时不需要额外的库文件。

```
# 1. 编译目标文件
gcc -c math_utils.c -o math_utils.o

# 2. 打包为静态库 (libmath.a)
ar rcs libmath.a math_utils.o

# 3. 链接静态库
gcc main.c -L. -lmath -o program
# -L. 表示在当前目录查找库，-lmath 表示链接 libmath.a
```

Listing 8.7: 静态库制作与使用 (Linux)

8.7.2 动态库/共享库 (Shared Library)

动态库在编译时仅记录引用信息，在程序运行时才加载到内存。多个程序可共享同一份动态库，节省内存和磁盘空间。

```
# 1. 编译位置无关代码 (Position Independent Code)
gcc -c -fPIC math_utils.c -o math_utils.o

# 2. 打包为动态库 (libmath.so)
gcc -shared -o libmath.so math_utils.o

# 3. 链接动态库
gcc main.c -L. -lmath -o program

# 4. 运行时指定库路径 (否则可能找不到 .so)
export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH
./program
```

Listing 8.8: 动态库制作与使用 (Linux)

8.8 从 Makefile 到 CMake

Makefile 是 Unix/Linux 系统下的经典构建工具，但语法较为晦涩。现代 C/C++ 项目广泛采用 **CMake**，它通过跨平台的 CMakeLists.txt 脚本生成对应平台的构建系统（如 Makefile、Ninja、Visual Studio 工程）。

```
cmake_minimum_required(VERSION 3.10)
project(MyProject C)

# 设置 C 标准
set(CMAKE_C_STANDARD 11)

# 添加可执行文件
add_executable(program src/main.c src/utils.c)

# 链接数学库
target_link_libraries(program m)
```

Listing 8.9: CMakeLists.txt 基础示例

CMake 构建流程:

- 1. 在项目根目录创建 CMakeLists.txt。
- 2. 运行 `cmake -S . -B build` 生成构建文件到 build/ 目录。
- 3. 运行 `cmake --build build` 执行编译。

8.9 未定义行为 (UB) 常见清单

C 语言为了追求极致性能，将许多边界情况定义为”未定义行为”（Undefined Behavior, UB）。编译器不对 UB 做任何保证，可能导致程序崩溃、静默错误或看似正常但逻辑错误的结果。以下是最高频的 UB 清单：

未定义行为	示例代码
有符号整数溢出	<code>int x = INT_MAX; x++;</code>
解引用空指针	<code>int *p = NULL; *p = 10;</code>
访问已释放内存 (Use-After-Free)	<code>free(p); *p = 5;</code>
数组越界访问	<code>int a[5]; a[5] = 10;</code>
修改字符串字面量	<code>char *s = "hello"; s[0] = 'H';</code>
同一表达式多次修改同一变量	<code>int i = 0; i = i++;</code>
函数返回值类型不匹配	声明返回 <code>int</code> 但实际返回 <code>float</code>

表 8.2: C 语言常见未定义行为 (UB) 清单

防御建议：始终开启编译器警告（`-Wall -Wextra`），使用静态分析工具（如 Clang-Tidy、Cppcheck），并在关键逻辑中加入 `assert` 断言。

附录 A C 标准库函数速查手册 (I/O、内存与字符串)

本章汇总 C 语言标准库中最常用的输入输出、内存管理与字符串处理函数。

A.1 C 标准库头文件总览

头文件	主要功能
<stdio.h>	标准输入输出、文件操作、格式化 I/O
<stdlib.h>	内存管理、数值转换、随机数、程序控制、排序查找
<string.h>	字符串操作、内存操作
<math.h>	数学函数（三角、指数、对数、取整等）
<ctype.h>	字符分类与大小写转换
<time.h>	时间与日期处理
<assert.h>	运行时断言调试
<errno.h>	错误码定义
<stdarg.h>	可变参数处理
<limits.h>	整型类型取值范围常量
<float.h>	浮点型精度与范围常量
<stddef.h>	常用类型定义（size_t, ptrdiff_t, NULL 等）
<setjmp.h>	非局部跳转（setjmp/longjmp）
<signal.h>	信号处理
<locale.h>	本地化设置
<stdbool.h>	布尔类型支持（C99）
<inttypes.h>	固定宽度整型与格式化宏（C99）
<stdint.h>	精确宽度整型与极限值（C99）
<complex.h>	复数运算支持（C99）
<tgmath.h>	泛型数学宏（C99）
<wchar.h>	宽字符 I/O 与操作（C95）
<wctype.h>	宽字符分类与转换（C95）
<fenv.h>	浮点环境控制（C99）
<iso646.h>	替代拼写宏（C95）

表 A.1: C 标准库头文件一览

A.2 <stdio.h> 标准输入输出库

A.2.1 格式化输入输出

函数原型	功能说明	返回值
<code>int printf(const char* fmt, ...)</code>	格式化输出到 stdout	成功输出的字符数
<code>int scanf(const char* fmt, ...)</code>	从 stdin 格式化输入	成功赋值的参数个数
<code>int fprintf(FILE* fp, const char* fmt, ...)</code>	格式化输出到文件	成功输出的字符数
<code>int fscanf(FILE* fp, const char* fmt, ...)</code>	从文件格式化输入	成功赋值的参数个数
<code>int sprintf(char* buf, const char* fmt, ...)</code>	格式化输出到字符串缓冲区	写入的字符数（不含 \0）
<code>int snprintf(char* buf, size_t n, const char* fmt, ...)</code>	安全版 sprintf，限制最大写入 n 字节	应写入的字符数
<code>int sscanf(const char* str, const char* fmt, ...)</code>	从字符串格式化输入	成功赋值的参数个数

表 A.2: stdio.h 格式化输入输出函数

A.2.2 字符与字符串 I/O

函数原型	功能说明	返回值
<code>int getchar(void)</code>	从 stdin 读取一个字符	读取的字符（EOF 表示结束）
<code>int putchar(int c)</code>	向 stdout 输出一个字符	输出的字符（EOF 表示失败）
<code>int fgetc(FILE* fp)</code>	从文件流读取一个字符	读取的字符（EOF 表示结束）
<code>int fputc(int c, FILE* fp)</code>	向文件流写入一个字符	写入的字符（EOF 表示失败）
<code>char* fgets(char* s, int n, FILE* fp)</code>	从文件流读取最多 n-1 个字符	s 指针（NULL 表示失败/结束）
<code>int fputs(const char* s, FILE* fp)</code>	向文件流写入字符串（不含 \0）	非负值成功，EOF 失败
<code>char* gets(char* s)</code>	已废弃：从 stdin 读取一行，无长度限制	s 指针（存在缓冲区溢出风险）

表 A.3: stdio.h 字符与字符串 I/O 函数

A.2.3 文件操作

函数原型	功能说明	返回值
<code>FILE* fopen(const char* path, const char* mode)</code>	以指定模式打开文件	FILE 指针（NULL 表示失败）
<code>int fclose(FILE* fp)</code>	关闭文件并刷新缓冲区	0 成功，EOF 失败
<code>size_t fread(void* ptr, size_t sz, size_t n, FILE* fp)</code>	从文件读取 n 个大小为 sz 的数据块	实际读取的块数
<code>size_t fwrite(const void* ptr, size_t sz, size_t n, FILE* fp)</code>	向文件写入 n 个大小为 sz 的数据块	实际写入的块数
<code>int fseek(FILE* fp, long off, int whence)</code>	移动文件位置指针	0 成功，非零失败
<code>long ftell(FILE* fp)</code>	获取当前文件位置偏移量	偏移量（-1L 表示失败）
<code>void rewind(FILE* fp)</code>	将文件位置指针重置到开头	无
<code>int feof(FILE* fp)</code>	检测是否到达文件末尾	非零表示已到末尾
<code>int ferror(FILE* fp)</code>	检测文件流是否发生错误	非零表示有错误
<code>int remove(const char* path)</code>	删除指定文件	0 成功，非零失败
<code>int rename(const char* old, const char* new)</code>	重命名文件	0 成功，非零失败
<code>void clearerr(FILE* fp)</code>	清除文件流的错误和 EOF 标志	无
<code>void setbuf(FILE* fp, char* buf)</code>	设置文件流的缓冲区	无
<code>int fflush(FILE* fp)</code>	刷新输出缓冲区，将数据写入文件	0 成功，EOF 失败

表 A.4: stdio.h 文件操作函数

A.2.4 其他 I/O 与错误处理

函数原型	功能说明	返回值
<code>int ungetc(int c, FILE* fp)</code>	将字符 c 推回流中	推入的字符（EOF 表示失败）
<code>void perror(const char* msg)</code>	将错误信息输出到 stderr	格式为 msg: strerror(errno)
<code>int setvbuf(FILE* fp, char* buf, int mode, size_t size)</code>	设置流的缓冲模式与缓冲区	0 成功，非零失败
<code>FILE* tmpfile(void)</code>	创建临时二进制文件（关闭自动删除）	FILE 指针（NULL 表示失败）
<code>char* tmpnam(char* s)</code>	生成唯一的临时文件名	指向文件名的指针

表 A.5: stdio.h 其他 I/O 与错误处理函数

A.3 <stdlib.h> 标准通用工具库

A.3.1 内存管理

函数原型	功能说明	返回值
<code>void* malloc(size_t size)</code>	分配 size 字节的未初始化内存	指向分配内存的指针（NULL 失败）
<code>void* calloc(size_t n, size_t sz)</code>	分配 n*sz 字节并初始化为 0	指向分配内存的指针（NULL 失败）
<code>void* realloc(void* ptr, size_t sz)</code>	重新调整已分配内存的大小	新指针（NULL 失败，原内存不变）
<code>void free(void* ptr)</code>	释放之前分配的内存	无

表 A.6: stdlib.h 内存管理函数

A.3.2 数值转换

函数原型	功能说明	返回值
<code>int atoi(const char* str)</code>	字符串转 int	转换后的整数值
<code>long atol(const char* str)</code>	字符串转 long	转换后的长整数值
<code>long long atoll(const char* str)</code>	字符串转 long long	转换后的 long long 值
<code>double atof(const char* str)</code>	字符串转 double	转换后的浮点值
<code>long strtol(const char* s, char** end, int base)</code>	字符串转 long, 支持指定进制	转换结果, end 指向首个无效字符
<code>double strtod(const char* s, char** end)</code>	字符串转 double	转换结果, end 指向首个无效字符

表 A.7: stdlib.h 数值转换函数

A.3.3 绝对值与除法

函数原型	功能说明	返回值
<code>int abs(int n)</code>	计算整数的绝对值	绝对值
<code>long labs(long n)</code>	计算 long 的绝对值	绝对值
<code>long long llabs(long long n)</code>	计算 long long 的绝对值 (C99)	绝对值
<code>div_t div(int num, int den)</code>	整数除法（返回商和余数）	div_t 结构体 {quot, rem}
<code>ldiv_t ldiv(long num, long den)</code>	long 整数除法	ldiv_t 结构体

表 A.8: stdlib.h 绝对值与除法函数

A.3.4 随机数与程序控制

函数原型	功能说明	返回值
<code>int rand(void)</code>	生成 [0,RAND_MAX] 范围内的伪随机数	随机整数值
<code>void srand(unsigned int seed)</code>	设置随机数种子	无
<code>void exit(int status)</code>	立即终止程序，刷新并关闭所有流	不返回
<code>void abort(void)</code>	异常终止程序，生成 SIGABRT 信号	不返回
<code>int atexit(void (*func)(void))</code>	注册程序正常终止时调用的函数	0 成功，非零失败
<code>int system(const char* cmd)</code>	执行系统命令	命令返回值

表 A.9: stdlib.h 随机数与程序控制函数

A.3.5 查找与排序

函数原型	功能说明	返回值
<code>void qsort(void* base, size_t n, size_t sz, int (*cmp)(const void*, const void*))</code>	对数组进行快速排序	无（原地排序）
<code>void* bsearch(const void* key, const void* base, size_t n, size_t sz, int (*cmp)(const void*, const void*))</code>	在已排序数组中二分查找	指向匹配元素的指针（NULL 未找到）

表 A.10: stdlib.h 查找与排序函数

A.3.6 多字节与宽字符转换

函数原型	功能说明	返回值
<code>int mblen(const char* s, size_t n)</code>	获取多字节字符的字节数	字节数（-1 表示无效）
<code>int mbtowc(wchar_t* wc, const char* s, size_t n)</code>	多字节字符转宽字符	字节数
<code>int wctomb(char* s, wchar_t wc)</code>	宽字符转多字节字符	字节数
<code>size_t mbstowcs(wchar_t* dest, const char* src, size_t n)</code>	多字节字符串转宽字符串	转换的宽字符数
<code>size_t wcstombs(char* dest, const wchar_t* src, size_t n)</code>	宽字符串转多字节字符串	转换的字节数

表 A.11: stdlib.h 多字节与宽字符转换函数

A.4 <string.h> 字符串处理库

A.4.1 字符串操作

函数原型	功能说明	返回值
<code>size_t strlen(const char* s)</code>	计算字符串长度（不含 \0）	字符个数
<code>char* strcpy(char* dst, const char* src)</code>	复制字符串（含 \0）	dst 指针
<code>char* strncpy(char* dst, const char* src, size_t n)</code>	最多复制 n 个字符	dst 指针
<code>char* strcat(char* dst, const char* src)</code>	拼接字符串到 dst 末尾	dst 指针
<code>char* strncat(char* dst, const char* src, size_t n)</code>	最多拼接 n 个字符	dst 指针
<code>int strcmp(const char* s1, const char* s2)</code>	字典序比较两个字符串	< 0 s1 小, = 0 相等, > 0 s1 大
<code>int strncmp(const char* s1, const char* s2, size_t n)</code>	最多比较前 n 个字符	同上
<code>char* strchr(const char* s, int c)</code>	查找字符 c 首次出现的位置	指向该位置的指针（NULL 未找到）
<code>char* strrchr(const char* s, int c)</code>	查找字符 c 最后一次出现的位置	指向该位置的指针（NULL 未找到）
<code>char* strstr(const char* haystack, const char* needle)</code>	查找子串首次出现的位置	指向子串起始位置的指针（NULL 未找到）
<code>char* strtok(char* str, const char* delim)</code>	按分隔符拆分字符串（内部维护状态）	下一个 token 指针（NULL 表示结束）
<code>char* strdup(const char* s)</code>	复制字符串（内部调用 malloc）	新分配的字符串指针

表 A.12: string.h 字符串操作函数

A.4.2 内存操作

函数原型	功能说明	返回值
<code>void* memcpy(void* dst, const void* src, size_t n)</code>	复制 n 字节（不可重叠）	dst 指针
<code>void* memmove(void* dst, const void* src, size_t n)</code>	复制 n 字节（可安全重叠）	dst 指针
<code>void* memset(void* ptr, int val, size_t n)</code>	将前 n 字节设置为 val	ptr 指针
<code>int memcmp(const void* s1, const void* s2, size_t n)</code>	比较前 n 个字节	< 0, = 0, > 0
<code>void* memchr(const void* s, int c, size_t n)</code>	在前 n 字节中查找值 c	指向该位置的指针（NULL 未找到）

表 A.13: string.h 内存操作函数

A.4.3 其他字符串操作

函数原型	功能说明	返回值
<code>char* strerror(int errnum)</code>	获取错误码对应的错误描述字符串	指向错误信息的指针
<code>size_t strspn(const char* s, const char* accept)</code>	计算 s 开头完全由 accept 中字符组成的长度	匹配的字符数
<code>size_t strcspn(const char* s, const char* reject)</code>	计算 s 开头不包含 reject 中字符的长度	匹配的字符数
<code>char* strpbrk(const char* s, const char* accept)</code>	查找 s 中首次出现 accept 中任意字符的位置	指向该位置的指针（NULL 未找到）
<code>int strcoll(const char* s1, const char* s2)</code>	根据当前 locale 比较字符串	< 0, = 0, > 0
<code>size_t strxfrm(char* dst, const char* src, size_t n)</code>	将字符串转换为 locale 相关的比较格式	转换后的长度

表 A.14: string.h 其他字符串操作函数

附录 B C 标准库函数速查手册 (数学、时间与字符)

本章汇总 C 语言标准库中的数学函数、时间处理以及字符分类与转换函数。

B.1 <math.h> 数学函数库

B.1.1 常用数学函数

函数原型	功能说明	返回值
<code>double sqrt(double x)</code>	平方根 $\sqrt{x}$	平方根值
<code>double pow(double x, double y)</code>	幂运算 x^y	幂值
<code>double exp(double x)</code>	自然指数 e^x	指数值
<code>double log(double x)</code>	自然对数 $\ln x$	对数值
<code>double log10(double x)</code>	常用对数 $\log_{10} x$	对数值
<code>double fabs(double x)</code>	绝对值 $ x $	绝对值
<code>double ceil(double x)</code>	向上取整 $\lceil x \rceil$	不小于 x 的最小整数
<code>double floor(double x)</code>	向下取整 $\lfloor x \rfloor$	不大于 x 的最大整数
<code>double round(double x)</code>	四舍五入	最接近的整数
<code>double fmod(double x, double y)</code>	浮点取余 $x - n \cdot y$	余数

表 B.1: math.h 常用数学函数

B.1.2 三角函数

函数原型	功能说明	返回值
<code>double sin(double x)</code>	正弦 (x 为弧度)	$\sin x$
<code>double cos(double x)</code>	余弦 (x 为弧度)	$\cos x$
<code>double tan(double x)</code>	正切 (x 为弧度)	$\tan x$
<code>double asin(double x)</code>	反正弦, 定义域 $[-1, 1]$	弧度值 $[-\pi/2, \pi/2]$
<code>double acos(double x)</code>	反余弦, 定义域 $[-1, 1]$	弧度值 $[0, \pi]$
<code>double atan(double x)</code>	反正切	弧度值 $(-\pi/2, \pi/2)$
<code>double atan2(double y, double x)</code>	双参数反正切 (判定象限)	弧度值 $(-\pi, \pi]$

表 B.2: math.h 三角函数

B.1.3 双曲函数

函数原型	功能说明	返回值
double sinh(double x)	双曲正弦 $\frac{e^x - e^{-x}}{2}$	双曲正弦值
double cosh(double x)	双曲余弦 $\frac{e^x + e^{-x}}{2}$	双曲余弦值
double tanh(double x)	双曲正切 $\frac{\sinh x}{\cosh x}$	双曲正切值

表 B.3: math.h 双曲函数

B.1.4 其他数学函数

函数原型	功能说明	返回值
double cbrt(double x)	立方根 $\sqrt[3]{x}$ (C99)	立方根值
double hypot(double x, double y)	计算 $\sqrt{x^2 + y^2}$ (避免溢出)	斜边长度
double log2(double x)	以 2 为底的对数 (C99)	对数值
double trunc(double x)	截断小数部分 (C99)	整数部分
double fmax(double x, double y)	返回两数中的较大值 (C99)	较大值
double fmin(double x, double y)	返回两数中的较小值 (C99)	较小值
double fma(double x, double y, double z)	乘加运算 $x \times y + z$ (C99)	单次舍入的乘加结果
int isnan(double x)	判断是否为 NaN (C99)	非零为真，零为假
int isinf(double x)	判断是否为无穷大 (C99)	非零为真，零为假
int isfinite(double x)	判断是否为有限值 (C99)	非零为真，零为假

表 B.4: math.h 其他数学函数

常量：M_PI ($\pi \approx 3.14159$)、M_E ($e \approx 2.71828$)。编译时需链接数学库：gcc -lm。

B.2 <ctype.h> 字符分类与转换库

函数原型	功能说明	返回值
<code>int isalpha(int c)</code>	是否为字母 (A-Z, a-z)	非零为真, 零为假
<code>int isdigit(int c)</code>	是否为数字 (0-9)	非零为真, 零为假
<code>int isalnum(int c)</code>	是否为字母或数字	非零为真, 零为假
<code>int isspace(int c)</code>	是否为空白字符 (空格、\t、\n 等)	非零为真, 零为假
<code>int isupper(int c)</code>	是否为大写字母	非零为真, 零为假
<code>int islower(int c)</code>	是否为小写字母	非零为真, 零为假
<code>int ispunct(int c)</code>	是否为标点符号	非零为真, 零为假
<code>int iscntrl(int c)</code>	是否为控制字符	非零为真, 零为假
<code>int isgraph(int c)</code>	是否为图形字符 (除空格外的可打印字符)	非零为真, 零为假
<code>int isxdigit(int c)</code>	是否为十六进制数字 (0-9, a-f, A-F)	非零为真, 零为假
<code>int isblank(int c)</code>	是否为空白字符 (空格或制表符, C99)	非零为真, 零为假
<code>int isprint(int c)</code>	是否为可打印字符	非零为真, 零为假
<code>int toupper(int c)</code>	转为大写	大写字符
<code>int tolower(int c)</code>	转为小写	小写字符

表 B.5: ctype.h 字符分类与转换函数

B.3 <time.h> 时间与日期库

函数原型	功能说明	返回值
time_t time(time_t* t)	获取当前日历时间 (秒数)	从 1970-01-01 起的秒数
double difftime(time_t t2, time_t t1)	计算两个时间之差	秒数差值
struct tm* localtime(const time_t* t)	将时间戳转为本地 时间结构体	tm 结构体指针
struct tm* gmtime(const time_t* t)	将时间戳转为 UTC 时间结构体	tm 结构体指针
time_t mktime(struct tm* tm)	将 tm 结构体转为 时间戳	time_t 值
char* asctime(const struct tm* tm)	将 tm 结构体转为 可读字符串	字符串指针
char* ctime(const time_t* t)	将时间戳转为可读 字符串	字符串指针
size_t strftime(char* s, size_t max, const char* fmt, const struct tm* tm)	按格式将 tm 转为 字符串	写入的字符数
clock_t clock(void)	获取程序运行至今 的 CPU 时间	clock_t 值 (除以 CLOCKS_PER_SEC 得 秒)

表 B.6: time.h 时间与日期函数

```
#include <stdio.h>
#include <time.h>

int main() {
    time_t now = time(NULL);
    struct tm* local = localtime(&now);
    printf("当前时间: %d-%02d-%02d %02d:%02d:%02d\n",
        local->tm_year + 1900, local->tm_mon + 1, local->tm_mday,
        local->tm_hour, local->tm_min, local->tm_sec);
    return 0;
}
```

Listing B.1: time.h 使用示例

结构体说明: struct timespec 包含 tv_sec (秒数) 和 tv_nsec (纳秒数), 用于高精度时间表示 (POSIX/C11)。

B.4 <complex.h> 复数运算库 (C99)

宏/函数	功能说明	返回值
<code>complex / _Complex</code>	复数类型关键字	支持 float/double/long double
<code>I</code>	虚数单位 $\sqrt{-1}$	复数常量
<code>double creal(double complex z)</code>	获取复数的实部	实部值
<code>double cimag(double complex z)</code>	获取复数的虚部	虚部值
<code>double cabs(double complex z)</code>	计算复数的模 $ z $	模值
<code>double carg(double complex z)</code>	计算复数的辐角（相位）	弧度值 $(-\pi, \pi]$
<code>double complex conj(double complex z)</code>	计算复共轭	共轭复数
<code>double complex csqrt(double complex z)</code>	复数平方根	平方根复数
<code>double complex cexp(double complex z)</code>	复数指数 e^z	复数值
<code>double complex clog(double complex z)</code>	复数自然对数	复数值
<code>double complex cpow(double complex x, double complex y)</code>	复数幂运算 x^y	复数值

表 B.7: complex.h 复数运算函数

B.5 <tgmath.h> 泛型数学宏库 (C99)

泛型宏	功能说明	行为
<code>sin(x), cos(x), tan(x)</code>	根据参数类型自动选择 float/double/long double 版本	传入 float 调用 <code>sinf</code> , 传入 double 调用 <code>sin</code>
<code>sqrt(x), pow(x, y)</code>	自动匹配数学函数变体	避免手动类型转换
<code>fabs(x), ceil(x), floor(x)</code>	自动选择绝对值、取整函数	支持整型与浮点型参数
<code>atan2(y, x), hypot(x, y)</code>	自动选择双参数函数版本	参数类型不同时提升为精度更高者

表 B.8: tgmath.h 泛型数学宏

B.6 <wchar.h> 宽字符 I/O 库 (C95)

函数原型	功能说明	返回值
wint_t fgetwc(FILE* fp)	从文件流读取一个宽字符	读取的宽字符 (WEOF 表示结束)
wint_t fputwc(wchar_t wc, FILE* fp)	向文件流写入一个宽字符	写入的宽字符 (WEOF 表示失败)
wchar_t* fgetws(wchar_t* s, int n, FILE* fp)	从文件流读取宽字符串	s 指针 (NULL 表示失败)
int fputws(const wchar_t* s, FILE* fp)	向文件流写入宽字符串	非负值成功, EOF 失败
wint_t getwchar(void)	从 stdin 读取一个宽字符	读取的宽字符
wint_t putwchar(wchar_t wc)	向 stdout 输出一个宽字符	输出的宽字符
size_t wcslen(const wchar_t* s)	计算宽字符串长度	宽字符个数
wchar_t* wcscpy(wchar_t* dst, const wchar_t* src)	复制宽字符串	dst 指针
int wcscmp(const wchar_t* s1, const wchar_t* s2)	比较宽字符串	< 0, = 0, > 0
wint_t btowc(int c)	单字节字符转宽字符	宽字符值
int wctob(wint_t wc)	宽字符转单字节字符	单字节值 (EOF 表示无法转换)

表 B.9: wchar.h 宽字符 I/O 与操作函数

B.7 <wctype.h> 宽字符分类库 (C95)

函数原型	功能说明	返回值
<code>int iswalnum(wint_t wc)</code>	是否为宽字母或数字	非零为真，零为假
<code>int iswalpha(wint_t wc)</code>	是否为宽字母	非零为真，零为假
<code>int iswdigit(wint_t wc)</code>	是否为宽数字	非零为真，零为假
<code>int iswspace(wint_t wc)</code>	是否为宽空白字符	非零为真，零为假
<code>int iswlower(wint_t wc)</code>	是否为宽小写字母	非零为真，零为假
<code>int iswupper(wint_t wc)</code>	是否为宽大写字母	非零为真，零为假
<code>wint_t towlower(wint_t wc)</code>	宽字符转小写	小写宽字符
<code>wint_t towupper(wint_t wc)</code>	宽字符转大写	大写宽字符
<code>wctype_t wctype(const char* property)</code>	获取字符属性描述符	属性类型
<code>int iswctype(wint_t wc, wctype_t desc)</code>	根据描述符检查宽字符属性	非零为真，零为假

表 B.10: wctype.h 宽字符分类与转换函数

附录 C C 标准库函数速查手册 (系统、类型与环境)

本章汇总 C 语言标准库中的系统工具、类型定义、错误处理与环境配置函数。

C.1 <assert.h> 断言库

宏/函数	功能说明	行为
assert(expression)	运行时断言检查	表达式为假时打印错误信息并调用 abort()
NDEBUG	宏定义	在 #include <assert.h> 前定义 NDEBUG 可禁用所有 assert

表 C.1: assert.h 断言宏

```
#include <assert.h>

int divide(int a, int b) {
    assert(b != 0); // 如果 b 为 0，程序终止并输出错误信息
    return a / b;
}
```

Listing C.1: assert 使用示例

C.2 <errno.h> 错误码库

宏/变量	功能说明	典型值
errno	全局错误码变量	由库函数在出错时设置
EDOM	数学函数定义域错误	如 sqrt(-1)
ERANGE	结果超出表示范围	如溢出
EAGAIN	资源暂时不可用	非阻塞 I/O 场景
EINVAL	无效参数	传入非法参数

表 C.2: errno.h 错误码

```
#include <stdio.h>
#include <errno.h>
#include <string.h>
#include <math.h>

int main() {
    errno = 0;
    double result = sqrt(-1.0);
    if (errno == EDOM) {
        printf("数学定义域错误: %s\n", strerror(errno));
    }
    return 0;
}
```

```
}

```

Listing C.2: errno 使用示例

C.3 <stdarg.h> 可变参数库

宏/类型	功能说明	用法
va_list	可变参数列表类型	用于声明参数遍历变量
va_start(ap, last)	初始化 va_list	last 是最后一个固定参数名
va_arg(ap, type)	获取下一个参数	type 为期望的参数类型
va_end(ap)	清理 va_list	在函数返回前调用

表 C.3: stdarg.h 可变参数宏

```
#include <stdio.h>
#include <stdarg.h>

double average(int count, ...) {
    va_list args;
    va_start(args, count);
    double sum = 0.0;
    for (int i = 0; i < count; i++) {
        sum += va_arg(args, double);
    }
    va_end(args);
    return sum / count;
}

int main() {
    printf("平均值: %.2f\n", average(4, 10.0, 20.0, 30.0, 40.0));
    return 0;
}
```

Listing C.3: 可变参数函数示例

C.4 <limits.h> 与 <float.h> 极限值常量

C.4.1 limits.h 整型极限

宏	含义	典型值
CHAR_BIT	char 的位数	8
CHAR_MIN / CHAR_MAX	char 最小/最大值	-128 / 127
INT_MIN / INT_MAX	int 最小/最大值	-2147483648 / 2147483647
LONG_MIN / LONG_MAX	long 最小/最大值	平台相关
LLONG_MIN / LLONG_MAX	long long 最小/最大值	$\approx \pm 9.22 \times 10^{18}$
UINT_MAX	unsigned int 最大值	4294967295

表 C.4: limits.h 常用宏常量

C.4.2 float.h 浮点型极限

宏	含义	典型值
FLT_MIN / DBL_MIN	float/double 最小正值	1.175×10^{-38} / 2.225×10^{-308}
FLT_MAX / DBL_MAX	float/double 最大值	3.402×10^{38} / 1.797×10^{308}
FLT_EPSILON / DBL_EPSILON	机器精度 ϵ	1.192×10^{-7} / 2.220×10^{-16}
FLT_DIG / DBL_DIG	有效十进制位数	6 / 15

表 C.5: float.h 常用宏常量

C.5 <stddef.h> 标准定义库

宏/类型	说明	典型值/用法
size_t	无符号整型，用于表示对象大小	sizeof 的返回类型
ptrdiff_t	有符号整型，用于表示指针差值	两个指针相减的结果类型
wchar_t	宽字符类型，用于存储宽字符	通常为 2 或 4 字节
NULL	空指针常量	通常定义为 <code>((void*)0)</code> 或 <code>0</code>
offsetof(type, member)	计算结构体成员相对于起始地址的偏移量	<code>offsetof(struct S, a)</code>

表 C.6: stddef.h 标准定义

C.6 <setjmp.h> 非局部跳转库

宏/类型	功能说明	返回值/行为
jmp_buf	数组类型，用于保存调用环境（寄存器状态、栈指针等）	供 setjmp 和 longjmp 使用
int setjmp(jmp_buf env)	保存当前执行环境到 env 中	直接调用返回 0；通过 longjmp 返回非零值
void longjmp(jmp_buf env, int val)	恢复到 env 保存的环境，并继续从 setjmp 处执行	setjmp 将返回 val（若 val 为 0 则返回 1）

表 C.7: setjmp.h 非局部跳转宏

C.7 <signal.h> 信号处理库

宏/函数	功能说明	典型值/行为
SIGINT	交互式注意信号（通常由 Ctrl+C 触发）	值为 2
SIGTERM	终止请求信号	值为 15
SIGSEGV	无效内存访问（段错误）	值为 11
SIG_DFL	默认信号处理动作	恢复系统默认行为
SIG_IGN	忽略信号	信号被丢弃，不执行任何操作
signal(int sig, void (*func)(int))	注册信号处理函数	返回旧的处理函数指针
raise(int sig)	向当前进程发送指定信号	成功返回 0，失败返回非零

表 C.8: signal.h 信号处理宏与函数

C.8 <locale.h> 本地化设置库

宏/函数	功能说明	典型值/行为
LC_ALL	设置所有本地化类别	涵盖数字、货币、时间、字符等
LC_COLLATE	影响字符串比较和排序规则	如 <code>strcoll</code> 的行为
LC_CTYPE	影响字符分类与转换函数	如 <code>isalpha</code> , <code>toupper</code>
LC_MONETARY	影响货币格式化	如 <code>localeconv</code> 返回的货币符号
LC_NUMERIC	影响数字格式化（如小数点符号）	如 <code>printf</code> 的小数点
LC_TIME	影响日期和时间格式化	如 <code>strftime</code> 的输出
<code>char* setlocale(int category, const char* locale)</code>	设置或查询当前程序的本地化环境	"" 表示使用系统环境变量
<code>struct lconv* localeconv(void)</code>	获取当前本地化环境的数字和货币格式信息	返回指向 <code>lconv</code> 结构体的指针

表 C.9: locale.h 本地化设置宏与函数

C.9 <stdbool.h> 布尔类型库 (C99)

宏/类型	说明
<code>bool</code>	扩展为 <code>_Bool</code> 类型
<code>true</code>	扩展为整数常量 1
<code>false</code>	扩展为整数常量 0
<code>__bool_true_false_are_defined</code>	扩展为 1，表示宏已定义

表 C.10: stdbool.h 布尔类型宏

C.10 <inttypes.h> 固定宽度整型库 (C99)

类型/宏	说明	典型值
int8_t / uint8_t	精确 8 位有符号/无符号整数	范围 ± 127 / $0 \sim 255$
int16_t / uint16_t	精确 16 位有符号/无符号整数	范围 ± 32767 / $0 \sim 65535$
int32_t / uint32_t	精确 32 位有符号/无符号整数	范围 $\pm 2 \times 10^9$ / $0 \sim 4 \times 10^9$
int64_t / uint64_t	精确 64 位有符号/无符号整数	范围 $\pm 9 \times 10^{18}$
PRId32 / PRIu32	printf 格式化固定宽度整型的宏	如 PRIu32 展开为 "u"
SCNd32 / SCNu64	scanf 格式化固定宽度整型的宏	如 SCNd32 展开为 "d"

表 C.11: inttypes.h 固定宽度整型与格式化宏

C.11 <stdint.h> 精确宽度整型库 (C99)

类型/宏	说明	典型值
int8_t / uint8_t	精确 8 位有符号/无符号整数	范围 $-128 \sim 127$ / $0 \sim 255$
int16_t / uint16_t	精确 16 位有符号/无符号整数	范围 $-32768 \sim 32767$
int32_t / uint32_t	精确 32 位有符号/无符号整数	范围 $\approx \pm 2 \times 10^9$
int64_t / uint64_t	精确 64 位有符号/无符号整数	范围 $\approx \pm 9 \times 10^{18}$
INT8_MIN / INT8_MAX	8 位整型最小/最大值	-128 / 127
INT32_MIN / INT32_MAX	32 位整型最小/最大值	-2147483648 / 2147483647
UINT_C(n) / INT_C(n)	生成指定宽度的整型常量	UINT32_C(10) 展开为 10U
intptr_t / uintptr_t	可容纳指针的有符号/无符号整型	用于指针与整数的安全转换

表 C.12: stdint.h 精确宽度整型与极限值

C.12 <fenv.h> 浮点环境库 (C99)

宏/函数	功能说明	返回值/行为
FE_DIVBYZERO	除零异常标志	浮点异常类型
FE_INEXACT	结果不精确异常标志	如 π 的近似值
FE_INVALID	无效操作异常标志	如 $\sqrt{-1}$
FE_OVERFLOW	上溢异常标志	结果超出最大值
FE_UNDERFLOW	下溢异常标志	结果低于最小正值
int feclearexcept(int excepts)	清除指定的浮点异常标志	0 成功
int fetestexcept(int excepts)	测试哪些异常标志被设置	被设置的标志位掩码
int feraiseexcept(int excepts)	手动触发浮点异常	0 成功
int fegetround(void)	获取当前浮点舍入模式	舍入模式常量
int fesetround(int round)	设置浮点舍入模式	0 成功, 非零失败
fenv_t	保存完整浮点环境的状态类型	供 fegetenv/fesetenv 使用

表 C.13: fenv.h 浮点环境控制函数

C.13 <iso646.h> 替代拼写宏库 (C95)

宏	替代运算符
and	&&
or	
not	!
bitand	&
bitor	
xor	^
compl	~
and_eq	&=
or_eq	=
xor_eq	^=
not_eq	!=

表 C.14: iso646.h 替代拼写宏